ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

DISEÑO Y DESARROLLO DE UNA APLICACIÓN SOFTWARE, EN EL CONTEXTO DEL ENFOQUE ÁGIL, COMO SOPORTE A LA REDUCCIÓN DEL NÚMERO DE ANIMALES DE COMPAÑÍA SIN HOGAR EN EL DISTRITO METROPOLITANO DE QUITO

DESARROLLO DEL MÓDULO DE ADOPCIÓN EN UNA APLICACIÓN MÓVIL CON IMPLEMENTACIÓN DE UN ECOSISTEMA DE RED SOCIAL

TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE INGENIERO/A EN SOFTWARE

ERICK STEVEN MUÑOZ ÑAUPARI

erick.munoz@epn.edu.ec

DIRECTOR: CARLOS EFRAIN IÑIGUEZ JARRIN

carlos.iniguez@epn.edu.ec

DMQ, febrero 2024

CERTIFICACIONES

Yo, ERICK STEVEN MUÑOZ ÑAUPARI declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

ERICK STEVEN MUÑOZ ÑAUPARI

Certifico que el presente trabajo de integración curricular fue desarrollado por ERICK STEVEN MUÑOZ ÑAUPARI, bajo mi supervisión.

Carlos Efraín Iñiguez Jarrín
DIRECTOR

DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

ERICK STEVEN MUÑOZ ÑAUPARI

CARLOS EFRAÍN IÑIGUEZ JARRÍN

DEDICATORIA

A mis maravillosos padres, Henry Muñoz y Paulina Ñaupari, y a mis queridos hermanos Adrian Muñoz y Pamela Muñoz, dedico esta tesis con profundo agradecimiento. Su apoyo incondicional ha sido el cimiento de mi trayectoria académica, y cada logro alcanzado es un reflejo de su amor y respaldo constante.

En particular, deseo mencionar lo importante que fueron mis padres, por su dedicación y sacrificio. Su apoyo ha sido esencial en mi camino, y valoro enormemente el papel fundamental que cada uno ha desempeñado en mi educación y mi vida.

A mis amigos Leo, Landy, Renato, Sebastian, Santiago y Dome, quienes han compartido risas y noches de videojuegos, agradezco su amistad y momentos de distracción que han aligerado la carga de los desafíos académicos.

Quisiera también dedicar este logro a mi fiel compañero perruno, Atom, quien con sus travesuras y lealtad inquebrantable ha sido una constante fuente de inspiración. Su alegría y presencia han aligerado los momentos más desafiantes.

Este logro es resultado del amor y apoyo de mi familia y amigos. A todos ustedes, mi más sincero agradecimiento.

Con gratitud,

Erick Muñoz

ÍNDICE DE CONTENIDO

CERTIFICACIONES I
DECLARACIÓN DE AUTORÍA II
DEDICATORIA III
ÍNDICE DE CONTENIDO IV
ÍNDICE DE FIGURAS VII
ÍNDICE DE TABLAS IX
RESUMEN X
ABSTRACT XI
1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO 1
1.1 Objetivo general 2
1.2 Objetivos específicos 2
1.3 Alcance 2
1.4 Marco teórico 3
Arquitectura limpia 3
Desarrollo Front-End 5
1. TypeScript 5
2. React 6
3. React Native 6
4. Expo 7
Entorno de desarrollo 8
1. Visual Studio Code 8
2. Git 8
Diseño de interfaz 9
1. Diseño centrado en el usuario 9
2. Figma 10
Herramientas de gestión de proyectos 11
1. Scrum 11
1.1. Roles de Scrum 11
1.2. Actividades y artefactos de Scrum 12
2. Jira 13
2 METODOLOGÍA 14
2.1 Sprint 0 15
Comprender las necesidades de los usuarios mediante la aplicación del DCU 15
1. Análisis de contexto de uso 15
2. Especificación de requisitos 16
2.1. Perfiles Persona 16
2.2. Análisis de tareas 17
3. Generación de soluciones de diseño 18
4. Evaluación de soluciones 19
Definición de herramientas y tecnologías de desarrollo 20
1. Elección de tecnología para el desarrollo de la aplicación 20

2. Elección de herramienta de control de versiones 20
3. Elección de Editor de código 20
4. Elección de Herramientas de gestión de proyectos 20
Definición de Arquitectura de software 21
Planificación de SCRUM y configuración inicial de herramientas 21
2.2 Sprint 1 21
2.2.1. Introducción 22
2.2.2. Planificación 22
2.2.3. Implementación 23
2.2.4. Revisión 26
2.2.5. Retrospectiva 28
2.3 Sprint 2 28
2.3.1. Introducción 28
2.3.2. Planificación 28
2.3.3. Implementación 30
2.3.4. Revisión 31
2.3.5. Retrospectiva 33
2.4 Sprint 3 35
2.4.1. Introducción 35
2.4.2. Planificación 35
2.4.3. Implementación 36
2.4.4. Revisión 38
2.4.5. Retrospectiva 40
2.5 Sprint 4 40
2.5.1. Introducción 40
2.5.2. Planificación 40
2.5.3. Implementación 41
2.5.4. Revisión 43
2.5.5. Retrospectiva 45
2.6 Sprint 5 45
2.6.1. Introducción 45
2.6.2. Planificación 45
2.6.3. Implementación 46
2.6.4. Revisión 48
2.6.5. Retrospectiva 50
2.7 Sprint 6 51
2.7.1. Introducción 51
2.7.2. Planificación 51
2.7.3. Implementación 52
2.7.4. Revisión 52
2.7.5. Retrospectiva 53
3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES 53
3.1 Resultados 53
Pruebas de usuario 53
Satisfacción de usuario 55

3.2 Conclusiones 57
3.3 Recomendaciones 58
4 REFERENCIAS BIBLIOGRÁFICAS 59
5 ANEXOS 60
ANEXO I. Tabla comparativa de herramientas de desarrollo Frontend recopiladas de [25], [26], [27], [28], [29]. 60
ANEXO II. Tabla comparativa de herramientas de Control de Versiones recopiladas de [16], [30], [31]. 61
ANEXO III. Tabla comparativa de Editores/IDEs recopilados de [32], [33], [34], [35], [36] 62
ANEXO V. Protocolo para Pruebas de Usabilidad - PAWSITIVE 63
ANEXO VI. Enlace a la documentación centralizada - Notion 65
ANEXO VII. Enlace al prototipo Figma refinado 65
ANEXO VIII. Cuestionario SUS: PAWSITIVE-QUITO 65
ANEXO IX. Encuesta sobre adopción, bienestar animal y tenencia responsable en el DMQ. 66
ANEXO X. Manual de instalación de PAWSITIVE-QUITO 71

ÍNDICE DE FIGURAS

Figura 1. Arquitectura Limpia (imagen adaptada de [10]) 4

Figura 2. Arquitectura de React Native para multiplataforma (tomada de [13]).....7

Figura 3. Diagrama de fases del DCU (tomada de [18]) 10

Figura 4. Ejemplo de mapeo de los sujetos encuestados.....16

Figura 5. Perfil persona primaria 16

Figura 6. Perfil persona secundaria.....17

Figura 7. Perfil persona terciaria 17

Figura 8. Prototipo inicial interfaz Adopciones.....19

Figura 9. Arquitectura limpia para Sprint 1 24

Figura 10. Diseño refinado de la interfaz adopciones (a) y filtro de adopciones (b) 25

Figura 11. Interfaces en el Sprint 1: adopciones (a) y filtro de adopciones (b) 27

Figura 12. Interfaz en sprint 1: formulario para agregar publicación de adopción.27

Figura 13. Arquitectura limpia para Sprint 2 30

Figura 14. Interfaces en Sprint 2: modal para marcar como favorito (a) y favoritos (b) 32

Figura 15. Interfaz en Sprint 2: formulario para agregar publicación (a) y publicación realizada (b) 32

Figura 16. Interfaces en Sprint 2: formulario para agregar nueva experiencia (a) y publicaciones de experiencias (b) 33

Figura 17. Ejemplo de tablero sprint retrospective, sprint 2.....34

Figura 18. Arquitectura limpia para Sprint 3 37

Figura 19. Interfaces en Sprint 3: registro de aptitud (a), sección de comentarios (b) y aptitud de adoptantes (c) 39

Figura 20. Interfaz en Sprint 3: cambio de botones en A, B y C y visibilidad de favoritos en D.....39

Figura 21. Arquitectura limpia para sprint 4 42

Figura 22. Interfaces en Sprint 4: Modal para llamadas (a) y aplicación de llamada con el número (b) 43

Figura 23. Interfaces en Sprint 4: like en adopciones en A (a) y sección de comentarios (b) 44

Figura 24. Interfaces en Sprint 4: compartir publicación a aplicaciones externas (izquierda) y perfil de usuario (derecha) 44

Figura 25. Arquitectura limpia para sprint 5 47

Figura 26. Interfaces en Sprint 5: inicio de sesión (a) y registro de usuario (b) 49

Figura 27. Interfaces en Sprint 5: botón para editar datos en A (a) y formulario de datos de usuario (b) 49

Figura 28. Interfaz en Sprint 5: organizaciones 50

Figura 29. Puntaje promedio por pregunta - Cuestionario SUS 55

Figura 30. Representación de resultados SUS (tomada de [24]) 57

ÍNDICE DE TABLAS

Tabla 1. Ejemplo de Matriz OTI para persona primaria 18

Tabla 2. Historias de Usuario Sprint 1. 22

Tabla 3. Tabla comparativa de esfuerzo sprint 1 28

Tabla 4. Historias de Usuario Sprint 2. 29

Tabla 5. Tabla comparativa de esfuerzo sprint 2 33

Tabla 6. Product Backlog Items sprint 3. 35

Tabla 7. Tabla comparativa de esfuerzo sprint 3 40

Tabla 8. Product Backlog Items sprint 4. 41

Tabla 9. Tabla comparativa de esfuerzo sprint 4 45

Tabla 10. Product Backlog Items sprint 5. 46

Tabla 11. Tabla comparativa de esfuerzo sprint 5 50

Tabla 12. Product Backlog Items sprint 6 51

Tabla 13. Tabla comparativa de esfuerzo sprint 6 53

Tabla 14. Tareas para pruebas de usuario.....53

Tabla 15. Resumen de comentarios categorizados por áreas de mejora 54

Tabla 16. Puntaje SUS por participante 56

RESUMEN

El módulo de la aplicación móvil que se describe en el presente documento pretende ser un soporte a la reducción de animales de compañía (perros y gatos) sin hogar en el distrito metropolitano de Quito. Dicho módulo se basa en un enfoque de red social especializada en el contexto de adopción y experiencias con animales de compañía que busca promover la adopción y mitigar el abandono.

La aplicación facilita la publicación de información relevante sobre animales en adopción y experiencias exitosas de adopción. Permite interacciones sociales con estas publicaciones y proporciona herramientas para identificar las aptitudes del adoptante. Además, establece canales de comunicación con usuarios que proponen animales en adopción. La implementación se realiza en una plataforma móvil que permite el acceso en cualquier momento desde un dispositivo smartphone con Android permitiendo atraer interesados y captar la atención a las personas dentro del segmento de adopción.

El desarrollo del módulo combina varias herramientas, como el Diseño Centrado en el Usuario, SCRUM, el enfoque de Arquitectura Limpia y React Native.

PALABRAS CLAVE: adopción de animales, React Native, arquitectura limpia, red social, animales de compañía, Android, SCRUM.

ABSTRACT

The module of the mobile application described in this document aims to support the reduction of homeless pets (dogs and cats) in the metropolitan district of Quito. This module is based on a specialized social network approach in the context of pet adoption and experiences, aiming to promote adoption and mitigate abandonment.

The application facilitates the publication of relevant information about animals available for adoption and successful adoption experiences. It allows social interactions with these posts and provides tools to identify the adopter's skills. Additionally, it establishes communication channels with users proposing animals for adoption. The implementation is carried out on a mobile platform, enabling access at any time from an Android smartphone, attracting interested parties and capturing the attention of people within the adoption segment.

The module's development combines various tools such as User-Centered Design, SCRUM, the Clean Architecture approach, and React Native.

KEYWORDS: pet adoption, React Native, clean architecture, social network, Companion animals, Android, SCRUM.

1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO

Según el censo realizado por la Unidad de Bienestar Animal (UBA) en marzo del 2022, en el Distrito Metropolitano de Quito existen 183 perros y 131 gatos abandonados por cada kilómetro cuadrado [1]. Los animales de compañía (perros y gatos) en situación de abandono generan riesgos a la salud pública y al ecosistema tales como, excretas en vías públicas, agresividad o mordeduras, esparcimiento de basura, accidentes de tráfico y enfermedades zoonóticas [2].

El abandono frecuente de animales de compañía está relacionado a factores socioeconómicos de las personas o desconocimiento sobre la tenencia responsable de estos animales [2]. La tenencia responsable se define, según la Ordenanza metropolitana No. 019 – 2020 del Distrito Metropolitano de Quito, como “el acto que le corresponde asumir a quien decide aceptar y mantener de forma temporal o permanente a un animal doméstico o de compañía, obligándose a cumplir libertades inherentes al Bienestar Animal… implica, además la obligación de adoptar todas las medidas necesarias para evitar que el animal doméstico o de compañía cause daños a las personas o a la propiedad de otros” [3]. Por ejemplo, desde el punto de vista de la tenencia responsable, asegurar una buena condición de vida de los animales de compañía implica responsabilidades relacionadas a su nutrición, vacunación, cuidados médicos frente a enfermedades y educación [4]. Asumir dichas responsabilidades depende de la situación socioeconómica de las personas; un factor que influye en el incremento de la tasa de abandono de animales de compañía. Otros factores son cambio de domicilio de las personas y problemas de comportamiento o salud en los animales de compañía [5].

En este contexto, promover la adopción es una posible solución al abandono. Para ello, se propone el análisis, diseño e implementación del módulo de adopción de animales de compañía en una plataforma móvil. Este módulo, busca apoyar el objetivo del proyecto macro: mitigar el creciente número de perros y gatos en las calles, además de promover el bienestar animal y evitar el abandono. La implementación del módulo seguirá un enfoque de red social especializada, es decir, el módulo brindará un espacio virtual donde los usuarios, interesados en dar en adopción o adoptar animales de compañía, podrán interactuar (difundir información, compartir experiencias de adopciones exitosas y establecer comunicación) con otros usuarios que comparten el mismo interés. Al usar una red social especializada como herramienta, es posible atraer interesados y captar la atención de las personas en el segmento de adopción [6].

Para llevar a cabo la gestión del desarrollo del módulo, se consideró el marco de trabajo ágil SCRUM debido a la flexibilidad que provee al equipo para adaptarse a lo largo del tiempo a las necesidades cambiantes del producto [7]. Por otro lado, considerando que el aplicativo móvil se constituye en un cliente de consumo de información y representa la interfaz de usuario con la que los usuarios interactuarán, su diseño estará guiado por la metodología Diseño Centrado en el Usuario (DCU). Al aplicar DCU es posible generar un prototipo de interfaz de usuario que pueden ser refinado a partir de las necesidades, limitaciones y características específicas de los usuarios.

1.1 Objetivo general

Desarrollar el módulo de adopción de una aplicación móvil mediante el enfoque de red social.

1.2 Objetivos específicos

1. Realizar una investigación de usuario aplicando técnicas del Diseño Centrado en el Usuario (DCU).
2. Diseñar y evaluar prototipos de interfaz de usuario.
3. Implementar el módulo a partir del consumo de las API(s) del backend.

1.3 Alcance

El alcance del componente incluye el diseño e implementación de las interfaces de usuario del módulo de adopción en una aplicación móvil aplicando un enfoque de red social para incentivar la adopción de animales de compañía del Distrito Metropolitano de Quito. La gestión de desarrollo del producto se realizará bajo las directrices del marco de trabajo SCRUM, aplicando sus artefactos y eventos. El diseño e implementación de las interfaces de usuario seguirá el proceso del Diseño Centrado en el Usuario consistente en las siguientes fases [8]:

- **Análisis del contexto de uso**: Comprender e interpretara los usuarios, sus objetivos, tareas y el entorno de uso.
- **Especificación de requisitos**: Identificar las funcionalidades y características del sistema que satisfagan las necesidades de los usuarios.
- **Generación de soluciones de diseño**: Explorar soluciones de diseño que cumplan con los requisitos.

- **Evaluación de las soluciones**: Evaluar las soluciones de diseño propuestas.

1.4 Marco teórico

En las próximas secciones, se examinarán los conceptos básicos, herramientas y metodologías integradas en el desarrollo del módulo de adopción en plataforma móvil. El objetivo es profundizar en la aplicación práctica de estos conceptos y justificar su aplicación en la construcción de una solución que satisfaga las necesidades de los usuarios.

Arquitectura limpia

La evolución de la arquitectura de software ha perseguido principalmente la creación de sistemas escalables y fáciles de mantener. En este contexto, la arquitectura limpia es un enfoque de desarrollo de software que enfatiza la importancia de separar las preocupaciones en el diseño de software, empleando una arquitectura en capas con reglas de dependencia específicas que dividen las responsabilidades (por ejemplo, reglas de negocio, interfaces y capa de usuario), lo que permite implementar características distintivas como [9]:

- **Independiente del framework:** posibilita la utilización de frameworks o librerías sin la necesidad de adaptar todo el sistema a ellos.
- **Testeable:** es posible realizar pruebas sobre las reglas de negocio, independiente de las interfaces de usuario.
- **Independiente de la interfaz de usuario:** es posible realizar la sustitución de una interfaz sin modificar las reglas de negocio.
- **Independiente de la base de datos:** al no estar acoplado estrictamente a un tipo de motor de base de datos, es posible intercambiarlo, por ejemplo, cambiar de Oracle a PostgreSQL.
- **Independiente de cualquier agente externo:** la capa de reglas de negocio opera de manera autónoma, sin depender de elementos externos como conexiones a APIs u otros sistemas.

La regla principal de la arquitectura limpia se basa en que las dependencias solo pueden provenir de capas internas, es decir, las funciones, clases, variables u otras entidades de software definidas en las capas externas no pueden ser usados en las capas internas [9]. En este sentido, el flujo de información entra por las capas más externas en dirección hacia las más internas, como lo muestra la **Figura *1***, donde la información fluye entre tres capas

principales (dominio, aplicación e infraestructura), desde la capa externa (infraestructura) hacia la interna (dominio). Es importante considerar que la capa externa de infraestructura se adapta al entorno de desarrollo y puede abarcar desde interfaces de usuario, consideradas como elementos de presentación, hasta bases de datos que representan la persistencia.

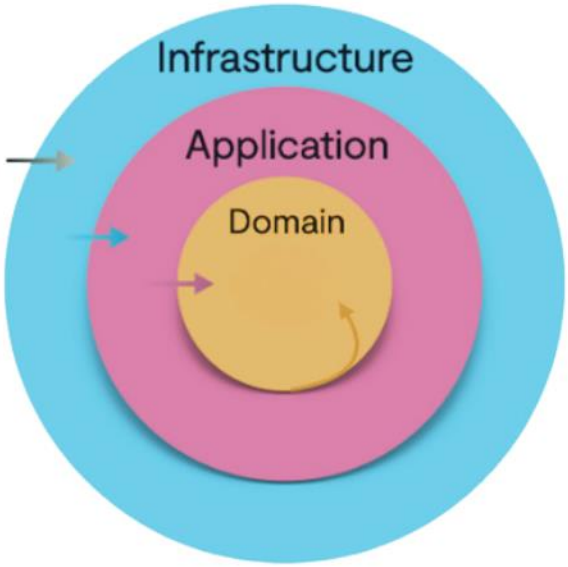


Figura 1. Arquitectura Limpia (imagen adaptada de [10])

Las tres capas simplificadas encapsulan diferente información según lo mencionado en [9]:

- Capa de Dominio. - También conocida como capa de entidades, encapsula las reglas fundamentales y estables del negocio, por medio de clases, estructuras de datos o funciones. Esta capa contiene los objetos de negocio, resguardando las reglas generales y fundamentales del negocio, asegurando que no se vean afectadas por cambios operativos de la aplicación. En el desarrollo del módulo, esta capa contiene las estructuras que definen las entidades del negocio y las abstracciones para establecer conexiones con elementos externos como el backend, librerías, frameworks y otros recursos asociados.

- Capa de Aplicación. - También conocida como capa de casos de uso, contiene reglas de negocio específicas que utilizan la capa de dominio. Por lo que, alteraciones en las funcionalidades o requisitos del sistema pueden incidir en el código de esta capa. En el desarrollo de este módulo, esta capa integra funciones y clases relacionadas a los requisitos del sistema, siendo representados como casos de uso, los cuales usan las abstracciones de la capa interna.

- Capa de Infraestructura. - Esta capa encapsula adaptadores, controladores y vistas, convirtiendo los datos al formato adecuado para las capas internas o para los frameworks utilizados. Además, establece la conexión con dichos frameworks, manteniéndolos en el exterior para reducir su impacto en el sistema. En el desarrollo de este módulo, esta capa contiene las interfaces de usuario y las implementaciones a las abstracciones definidas en dominio.

En la implementación del módulo bajo estudio se ha aplicado el enfoque de arquitectura limpia debido a su orientación hacia sistemas mantenibles y escalables. Su énfasis en la separación de responsabilidades permite flexibilidad y facilita la adaptación a cambios sin comprometer las reglas de negocio. Además, esta arquitectura minimiza el acoplamiento a frameworks, lo que asegura la sustituibilidad de componentes. La división en capas permite encapsular y preservar diferentes aspectos del sistema, asegurando que los cambios externos no afecten la lógica fundamental del negocio. Este enfoque se alinea con la necesidad de un sistema adaptable y resistente a cambios, elementos esenciales en el contexto de este módulo.

Desarrollo Front-End

1. TypeScript

En [11] se describe a TypeScript como una extensión de JavaScript que brinda mayor escalabilidad a las aplicaciones basadas en JavaScript. Aunque JavaScript se originó como un lenguaje de scripting, su uso se ha extendido a una variedad de aplicaciones web y multiplataforma. En entornos más complejos, es necesario implementar componentes reutilizables o aplicar conceptos como patrones de diseño. TypeScript ofrece un conjunto de herramientas que facilitan la implementación de programación orientada a objetos. Esto permite incorporar conceptos como herencia, encapsulamiento y abstracciones mediante elementos que se mencionan en [11] como, tipos estáticos, clases, interfaces, genéricos y módulos.

La validación de tipos se realiza mediante un compilador que convierte el código TypeScript a JavaScript [11]. Esta característica posibilita la detección de errores durante la compilación, en lugar de en tiempo de ejecución, agilizando la identificación y solución de errores o fallos en el código. Por lo tanto, en base a estas consideraciones, se eligió TypeScript como lenguaje de programación para el presente módulo.

2. React

React es una librería JavaScript/TypeScript creada por Meta para el manejo y creación de interfaces de usuario. El objetivo de esta librería es crear páginas webs reactivas minimizando el uso de código, para ello cuenta con herramientas como:

- **Componentes:** Son herramientas en React que envuelven tanto la lógica como la interfaz de un elemento específico, permitiéndole ser identificado mediante una etiqueta. Estos pueden ser componentes funcionales, creados con funciones, o componentes de clase, elaborados a partir de clases.
- **Hooks:** Funciones introducidas por React para ofrecer a los componentes funcionales capacidades similares a las de los componentes de clase. Estas funciones permiten "enganchar" funcionalidades específicas, como el manejo de estado, a un componente funcional.

La tecnología de React también es posible trasladarla a ambientes multiplataforma (IOS y Android) por medio de frameworks, como React Native.

3. React Native

Es un framework basado en la librería React de Meta, se centra en el desarrollo de aplicaciones móviles que permite la implementación de interfaces de usuario nativas al vincular la lógica de React con las APIs de renderizado nativo como Object-C para iOS y Java para Android a través de un puente [12]. De esta forma, es posible que la aplicación tenga acceso a componentes nativos y a características específicas de la plataforma. Al estar basado en React, React Native usa JSX o TSX, extensiones de JavaScript y TypeScript respectivamente, que emplean esquemas XML. Además, permite aprovechar componentes, hooks y diversas librerías compatibles con React. La documentación oficial de Meta define la arquitectura como se muestra en la **Figura *2***, donde Fabric es un código en C++ que funciona a manera de puente entre la tecnología de React y las plataformas móviles [13].

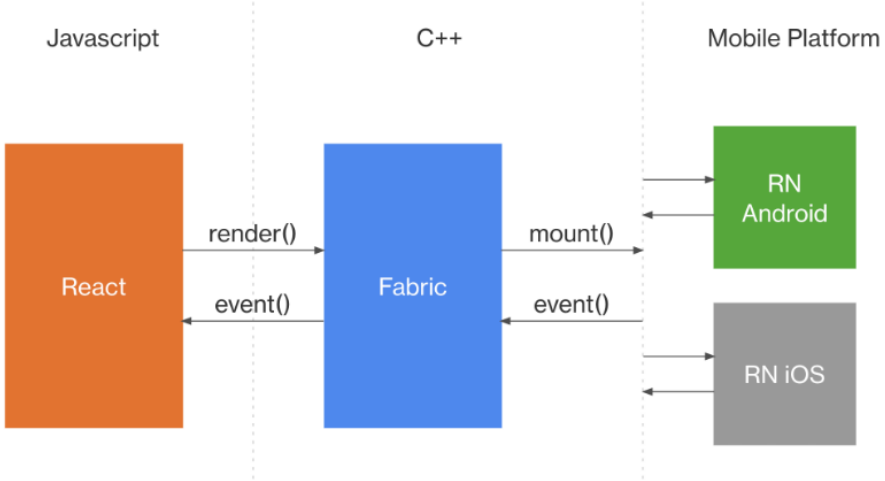


Figura 2. Arquitectura de React Native para multiplataforma (tomada de [13])

En resumen, React Native se destaca como una elección sólida para el desarrollo de aplicaciones móviles debido a su marco robusto y su soporte para JavaScript y Typescript. Cuenta con una extensa documentación soportada por Meta y una amplia gama de herramientas disponibles, permitiendo crear interfaces de usuario nativas. Esta flexibilidad junto con la potencia inherente de React, lo convierte en una opción viable para el presente módulo.

4. Expo

Expo se destaca como un entorno integral para React Native al ofrecer una gama completa de herramientas que simplifican el proceso de desarrollo, build y lanzamiento de aplicaciones. El término “build” hace referencia a la generación de una aplicación nativa para cada plataforma, ya sea IOS o Android. Algunas de las herramientas provistas por Expo se mencionan en [14]:

- **Expo Go:** aplicación Android y IOS, en la cual se podrá ejecutar el proyecto React Native durante el proceso de desarrollo, sin realizar un build, también permite la visualización de los cambios en tiempo real.
- **Expo CLI:** es una herramienta de línea de comando que permite ejecutar un servidor local sobre el cual se ejecutará la aplicación en Expo Go, realizar builds y publicar proyectos.
- **Expo SDK:** El SDK permite acceso a varias APIs nativas, como cámara, ubicación, acelerómetro, notificaciones entre otras funcionalidades.

Expo se presenta como una poderosa herramienta en el desarrollo de React Native, simplificando tareas fundamentales como desarrollo, pruebas y despliegue. Optar por Expo

al crear una aplicación móvil en React Native representa una elección inteligente que optimiza múltiples procesos, potenciando así la eficiencia del desarrollo del presente módulo.

Entorno de desarrollo

1. Visual Studio Code

Visual Studio Code (VSC), es un editor de código de Microsoft, que destaca por su ligereza, agilidad y rapidez en comparación con los entornos de desarrollo integrados (IDE), mientras ofrece una amplia gama de extensiones para escribir y ejecutar código en múltiples lenguajes, integrando funciones como la gestión de contenedores y la conexión con máquinas virtuales, incluyendo el Windows Subsystem for Linux [15].

Las principales características de VSC se mencionan en [15]:

- **Multiplataforma:** Esta disponible para Windows, Linux y macOS.
- **IntelliSense:** Esta función optimiza la escritura y edición del código fuente al ofrecer autocompletado, sugerencias, resaltado de sintaxis y la identificación de errores. Además, su potencial puede ampliarse mediante el uso de extensiones.
- **Depuración:** Incluye un sistema de depuración que facilita la identificación y corrección de errores en el código.
- **Control de versiones:** Ofrece una sólida integración con Git, lo que permite realizar operaciones como git diff, organizar archivos y ejecutar funciones de Git, como commits, push y pull, entre otras.
- **Extensiones:** Uno de sus puntos más destacados es la amplia comunidad de desarrolladores que respalda una extensa biblioteca de extensiones. Estas agregan funcionalidades y permiten personalizar el editor, cambiar temas, esquemas de colores o integraciones con servicios y frameworks.

2. Git

Git, concebido por Linus Torvalds en 2005, es un sistema de control de versiones moderno y consolidado, distribuido y enfocado en el rendimiento, la seguridad y la flexibilidad en su diseño, permitiendo que cada réplica del repositorio contenga el historial completo en un ordenador [16].

Algunos conceptos básicos son mencionados en [16]:

- **Repositorio:** Es el espacio de trabajo donde se aloja un proyecto gestionado con Git. Aquí se registra y guarda su historial de cambios.
- **Commit:** Representa una modificación en el código que un desarrollador "compromete". Durante este proceso, se detallan los cambios realizados y el motivo detrás de ellos.
- **Ramas:** Son líneas de desarrollo alternativas en un proyecto. La rama principal, generalmente llamada main o master, es la base. Para implementar cambios significativos o nuevas funciones, se crea una rama aparte, una réplica exacta de la rama principal, pero aislada. Este enfoque preserva la integridad del proyecto principal, evitando comprometerlo ante posibles errores. Si los cambios son exitosos, pueden fusionarse posteriormente con el proyecto principal.
- **Clon:** Se refiere a replicar el repositorio de manera local. Esto se realiza a menudo desde repositorios remotos para que cada desarrollador tenga su propia copia local del proyecto en su máquina.

En síntesis, la elección de Git como sistema de versionamiento para este módulo se fundamenta en su flexibilidad, potencia y la capacidad de facilitar el trabajo colaborativo mediante su sistema distribuido.

Diseño de interfaz

1. Diseño centrado en el usuario

El diseño centrado en el usuario (DCU) se basa en una metodología que pone al usuario como eje central en todos los procesos de diseño, con el fin de crear productos funcionales y satisfacer sus necesidades. Este enfoque sigue un proceso iterativo con fases interrelacionadas para adaptarse a las cambiantes necesidades del usuario [17]. La definición de estas fases se menciona en [17] y [8] :

- **Análisis de contexto de uso:** Se refiere a comprender las situaciones específicas en las que se emplea un sistema, observando cómo los usuarios interactúan en su entorno habitual. En el desarrollo de este módulo, en esta fase se realizó una encuesta a usuarios clave para conocer sus objetivos y limitaciones en el contexto de uso del presente módulo.
- **Especificación de requisitos:** Se trata de recopilar los requerimientos de los usuarios, considerando sus necesidades dentro del contexto de uso. Durante el

desarrollo de este módulo, se reconocieron y registraron actividades y atributos específicos que se transformaron en perfiles, escenarios y análisis de tareas.

- **Generación de soluciones de diseño:** Incluye explorar diversas opciones de diseño que se ajusten a los requisitos obtenidos. En el presente módulo, mediante herramientas de diseño y prototipado se creó una solución de diseño que abordaba las necesidades identificadas.
- **Evaluación de soluciones:** Consiste en analizar y valorar las soluciones propuestas para determinar su usabilidad en términos de efectividad, eficiencia y satisfacción del usuario. Durante el desarrollo de este módulo, se sometió la solución de diseño a pruebas con los usuarios clave para obtener retroalimentación, la cual fue analizada para mejorar la propuesta de diseño.

Al adoptar el DCU, como se muestra en la **Figura *3*** es posible realizar iteraciones que permite la adaptación constante del producto a las necesidades cambiantes de los usuarios. Este enfoque iterativo y centrado en el usuario asegura que el producto final sea funcional, relevante y valioso para sus usuarios.

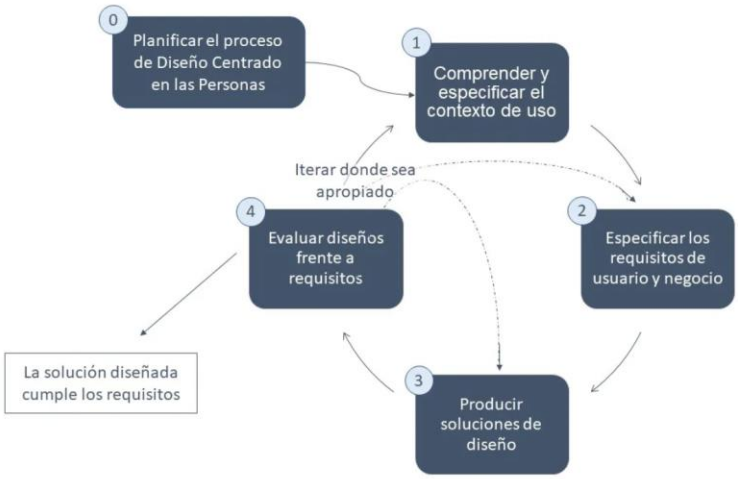


Figura 3. Diagrama de fases del DCU (tomada de [18])

2. Figma

Figma es una plataforma colaborativa de diseño de interfaz de usuario y prototipado cuyo enfoque central es simplificar y agilizar la creación de maquetas, prototipos y diseños para diversas aplicaciones, independientemente de la plataforma en la que operen. Figma opera en la nube y se destaca por su capacidad para crear una amplia gama de diseños, desde wireframes hasta prototipos animados.

Entre las principales características de Figma están: es multiplataforma, interfaz intuitiva, capacidad para crear diseños responsivos, amplia biblioteca de componentes y plantillas, integraciones con plugins y extensiones y trabajo colaborativo en tiempo real [19].

Al analizar sus características, Figma emerge como una solución valiosa para la implementación del Diseño Centrado en el Usuario DCU. Sus capacidades de diseño colaborativo, creación de prototipos interactivos y maquetación de interfaces de usuario permiten una iteración ágil y la participación continua de los usuarios. Esta herramienta posibilita la visualización temprana de la experiencia del usuario, fomentando la retroalimentación anticipada y la validación de conceptos. Al ofrecer un espacio para la experimentación con diseños y flujos de interacción, Figma se convierte en un punto de partida esencial para la planificación y el diseño detallado de la aplicación, facilitando así un proceso de desarrollo más enfocado y centrado en las necesidades del usuario.

Herramientas de gestión de proyectos

1. Scrum

Scrum se define como un marco ágil para la gestión de proyectos complejos [20]. Scrum estructura y gestiona el trabajo basándose en valores, principios y prácticas específicas, enfocándose en la construcción de un "product backlog" que lista características priorizadas clave para el éxito del producto, asegurando así una concentración continua en las tareas críticas y prioritarias [21].

Durante iteraciones de una semana a un mes, un equipo multifuncional y autoorganizado trabaja en el diseño, construcción y pruebas de funcionalidades completas, seleccionando previamente un subconjunto de tareas del "product backlog" para cada ciclo; al final de cada uno, se revisan estas características con las partes interesadas para recibir retroalimentación, buscando avances tangibles en el producto en desarrollo [21].

1.1. Roles de Scrum

Para el funcionamiento de un equipo de Scrum es necesario tres roles específicos mencionados en [21]:

- **Product Owner (PO):** Es el encargado de determinar las características y funciones a desarrollar, ofreciendo una visión definida del objetivo a alcanzar. En esencia, este rol es crucial para garantizar el éxito de la solución propuesta.
- **Scrum Master:** Desempeña el papel de facilitador del uso efectivo de Scrum. Su función radica en guiar al equipo Scrum para que adopte plenamente los principios,

valores y prácticas de este marco de trabajo. Actúa como líder, fomentando el máximo rendimiento del equipo y promoviendo un entorno que impulse su productividad.

- **Equipo de desarrollo:** El equipo de desarrollo está compuesto por una variedad de roles, incluyendo desarrolladores de software, diseñadores de interfaz, testers, administradores de bases de datos, entre otros profesionales. Este equipo debe funcionar como una entidad autoorganizada y multifuncional, ya que su responsabilidad principal recae en la construcción, prueba y despliegue exitoso del producto.

1.2. Actividades y artefactos de Scrum

Scrum provee de ciertas actividades y artefactos [21] .

- **Product Backlog:** En Scrum se prioriza el valor del producto. Por ello, el Product Owner, con el equipo Scrum y las partes interesadas, deben establecer una lista secuencial de tareas priorizadas en base a la visión del PO y el valor que aportan. Esta lista puede incluir también tareas para resolver defectos o mejoras técnicas.

- **Sprints:** En Scrum, los "sprints" son iteraciones que se realizan en un rango de tiempo que puede extenderse hasta un mes. Cada finalización de sprint debe resultar en la entrega de un producto que aporte valor tangible al cliente.

- **Sprint Planning:** Es una sesión en la que el equipo Scrum establece el propósito del próximo sprint y selecciona las tareas del "product backlog" alineadas con el objetivo.

- **Daily Scrum:** Es una reunión del equipo de desarrollo en Scrum diseñada para agilizar y flexibilizar el flujo de trabajo. En ella, cada miembro del equipo menciona los avances desde el último Daily Scrum, define las tareas próximas y expone cualquier obstáculo que pueda limitar el progreso.

- **Done:** En Scrum, para establecer lo que constituye el resultado final de un sprint, el equipo debe definir claramente los criterios por los cuales se considera al resultado final como "terminado". El incremento generado durante el sprint debe cumplir con estos criterios establecidos, lo que agrega confianza y calidad al trabajo realizado.

- **Sprint Review:** Representa una reunión crucial en Scrum, donde el equipo de desarrollo y las partes interesadas se encuentran para lograr una sincronización

bidireccional y un intercambio de información. Esta reunión permite obtener una visión global del progreso del producto y alinear su dirección comercial con las necesidades de los clientes. Es una oportunidad valiosa para retroalimentar y adaptar el producto en función de la retroalimentación recibida.

- **Sprint Retrospective:** Se lleva a cabo típicamente después del Sprint Review y antes de la planificación del próximo sprint, marcando el cierre de cada iteración. Esta reunión se convierte en la oportunidad para inspeccionar y ajustar el proceso de desarrollo del producto. Su objetivo principal radica en discutir las áreas de mejora dentro de Scrum y las prácticas técnicas relacionadas. A partir de esta evaluación, se establecen acciones concretas, consolidando un compromiso hacia la mejora continua del proceso en Scrum y la optimización constante de las prácticas de desarrollo.

La elección de SCRUM para gestionar el módulo bajo estudio se fundamenta en las directrices de Cynefin; un marco que evalúa la complejidad de los proyectos y orienta las decisiones en base a cinco dominios: simple, caótico, complejo, complicado y desordenado [22]. Específicamente, en el ámbito "complejo", los proyectos se caracterizan por arrojar resultados impredecibles, por lo cual se requiere una adaptación gradual para mitigar los problemas durante el ciclo del proyecto. Cynefin sugiere emplear SCRUM para gestionar estos proyectos “complejos”, ya que permite una adaptabilidad progresiva, manejo controlado de cambios y alineación con un enfoque iterativo.

Considerando la clasificación de proyectos en Cynefin, el módulo bajo estudio se sitúa en el dominio "complejo" debido a las necesidades cambiantes de los usuarios, lo que conlleva a resultados impredecibles. Por esta razón, se implementa SCRUM en la gestión del proyecto, posibilitando al equipo ajustarse a los constantes cambios en las necesidades de los usuarios.

2. Jira

Jira se introdujo en 2002 como una plataforma para el seguimiento de incidencias y la gestión de proyectos ágiles en equipo [23]. Ofrece funcionalidades para planificar, supervisar, publicar y brindar soporte en el desarrollo de software [23]. Facilita la centralización de información a lo largo del ciclo de vida de desarrollo y fomenta la conectividad entre equipos de desarrollo, lo que se traduce en una mayor productividad, coordinación y comunicación fluida entre ellos [23]. Jira cuenta con algunas características específicas mencionadas en [23] que facilitan la implementación de un enfoque ágil en la gestión de proyectos:

- **Paneles:** Facilita la administración del trabajo a través de paneles en tableros Scrum o Kanban.
- **Cronograma:** Permite realizar un seguimiento del trabajo y mantener al equipo sincronizado mediante esta función.
- **Informes:** Ofrece una variedad de informes listos para usar según el flujo de trabajo realizado.
- **Integraciones:** Amplía sus funcionalidades con más de 3000 aplicaciones e integraciones, como GitHub, Slack, entre otras.

Al analizar las capacidades de Jira en relación con sus características y su perfecta adaptabilidad al enfoque ágil, se evidencia que esta herramienta ofrece un ecosistema idóneo para optimizar el flujo de trabajo y la gestión del presente módulo.

2 METODOLOGÍA

En esta sección se detalla la implementación de Scrum a lo largo de 6 sprints, cada uno presentado en detalle a través de cinco subsecciones:

- Introducción. – Aquí se establece el objetivo de cada sprint junto con las decisiones cruciales tomadas.
- Planificación. – Se aborda la actividad central de Scrum, el "Sprint Planning". Se identifican las historias de usuario (HU) seleccionadas específicamente para alinearse con el objetivo del sprint. Cada HU incluye las funcionalidades del sistema, la estimación de esfuerzo en la escala de Fibonacci, los criterios de aceptación y un identificador único.
- Implementación. – Aquí se explica cómo se abordó la resolución de las HU desde una perspectiva de diseño y desarrollo. Se mencionan pasos clave seguidos para generar resultados tangibles relacionados con las HU durante el sprint.
- Revisión. – Este apartado se corresponde con la reunión Scrum "Sprint Review" mencionada en la sección 1.2, en la cual se presenta el incremento resultante del sprint para obtener retroalimentación de las partes interesadas. Además, se contrasta la estimación inicial de esfuerzo con el esfuerzo efectivo aplicado, buscando alinearse con el objetivo de mejora continua.
- Retrospectiva. – En esta sección se aborda la reunión de “Sprint Retrospective” definida en la sección 1.2, donde se presentan las conclusiones clave extraídas del

autoanálisis del equipo Scrum durante el desarrollo de cada sprint, transformándolas en oportunidades de mejora para los próximos sprints.

2.1 Sprint 0

En este Sprint se abordaron dos actividades claves para el desarrollo de los subsecuentes Sprints:

1. Comprender las necesidades de los usuarios. – Considerando los objetivos específicos 1 y 2 de este documento, los cuales implican identificar las necesidades de los usuarios dentro del ámbito del presente módulo aplicando el DCU. Para ello, se plantea aplicar dos técnicas del DCU: 1) perfil Persona y 2) el análisis de tareas a través de la matriz OTI (matriz de objetivos, tareas e interfaces).

2. Análisis de herramientas para el desarrollo – Considerando el contexto del componente y las necesidades de los usuarios se hará la selección tecnologías y herramientas necesarias.

3. Definición de Arquitectura de software. – En base a las necesidades obtenidas en el DCU, se definirá una arquitectura de software adecuada.

4. Planificación SCRUM y configuración inicial de herramientas. – Considerando las herramientas de manejo de proyectos y los requisitos obtenidos, se definirá la planificación para las actividades y artefactos de SCRUM.

Comprender las necesidades de los usuarios mediante la aplicación del DCU

1. Análisis de contexto de uso

Para comprender e interpretar a los usuarios clave, sus objetivos y el contexto de uso del módulo bajo estudio, se desarrolló una encuesta definida en el

ANEXO IX, dirigida a 60 personas con un historial de adopción de uno o varios animales de compañía en el Distrito Metropolitano de Quito (DMQ). El objetivo de esta encuesta fue recolectar información sobre la percepción y conocimiento de las personas residentes en el DMQ con respecto a la adopción de perros y gatos, el bienestar animal y la tenencia responsable.

2. Especificación de requisitos

2.1. Perfiles Persona

En base a los resultados obtenidos en la encuesta definida en el

ANEXO IX, se realizó un mapeo de los encuestados en función de variables de comportamiento como se muestra en la **Figura *4***, para obtener los perfiles de las personas objetivo.

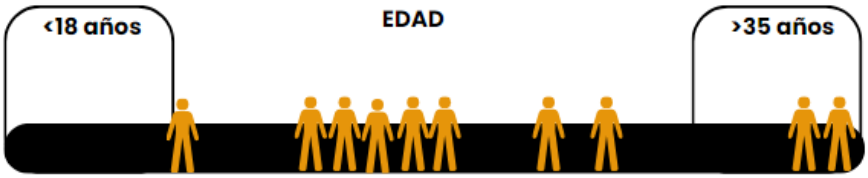


Figura 4. Ejemplo de mapeo de los sujetos encuestados

Tras el mapeo y análisis de los encuestados, se identificaron 3 perfiles Persona (primario, secundario y terciario). El perfil primario mostrado en la **Figura *5***, corresponde a una estudiante con empatía hacia los animales de compañía; está consciente del cuidado y bienestar que requieren y está interesada en fomentar la adopción. El perfil secundario mostrado en la **Figura *6***, representa a un hombre adulto que cuida a una mascota y valora la importancia de encontrarles un hogar adecuado. Por último, el perfil terciario mostrado en la **Figura *7***, está caracterizado por su amor hacia los animales.

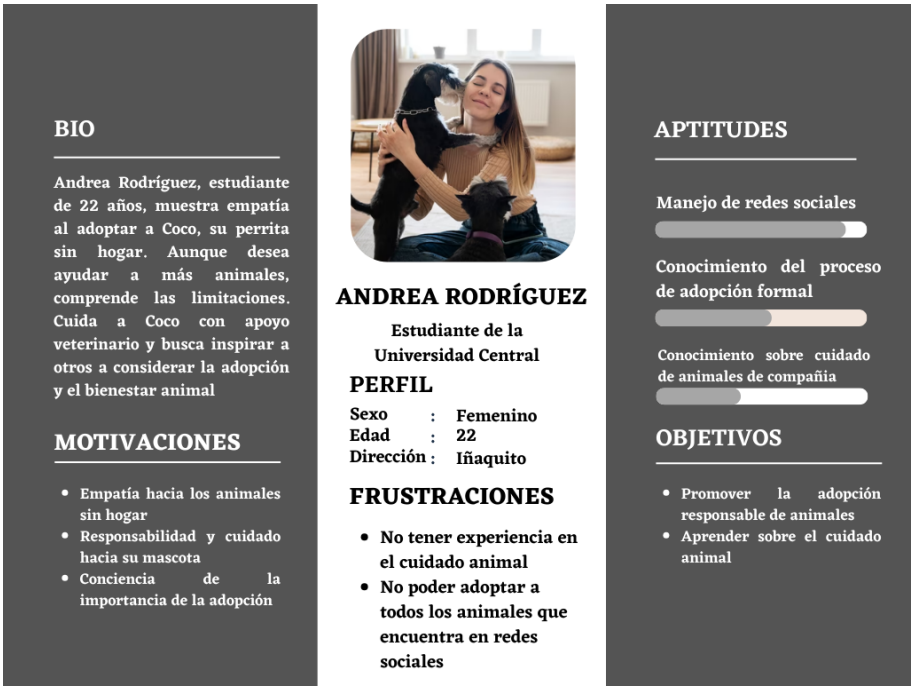


Figura 5. Perfil persona primaria

Figura 6. Perfil persona secundaria

Figura 7. Perfil persona terciaria

2.2. Análisis de tareas

Considerando la información de cada perfil Persona, se hizo el análisis de tareas con la matriz OTI, de esta forma, se establecieron los objetivos, las tareas asociadas a cada objetivo y las interfaces que permiten lograr dichas tareas, como se muestran en la **Tabla *1***.

Tabla 1. Ejemplo de Matriz OTI para persona primaria

OBJETIVOS	TAREAS	INTERFAZ
Promover la adopción responsable de animales	• Registrar usuario. • Crear publicaciones sobre su experiencia al adoptar. • Compartir publicaciones de animales en adopción • Interactuar con publicaciones (likes, comentarios).	• Registro de usuario. • Inicio de sesión. • Adopciones (contiene opción para agregar publicación de experiencia). • Formulario de publicación de experiencia. • Adopciones (contiene, por cada publicación, opción para compartir de forma externa). • Adopciones (contiene, por cada publicación, opción para hacer "like" y "comentar") • "Experiencias"

Los hallazgos aportados por la matriz OTI traducidos en tareas de valor para el usuario se han convertido en historias de usuario iniciales correspondientes a 4 épicas para la ejecución de SCRUM: Promoción de la adopción responsable, Soporte al Proceso de Adopción, Funcionalidades de usuario y perfil.

3. Generación de soluciones de diseño

Considerando los requisitos recopilados, se elaboró una propuesta de diseño mediante un prototipo. Utilizando la herramienta Figma, se llevó a cabo la navegación inicial de las diferentes interfaces relacionadas con los requisitos clave. La **Figura *8*** presenta un ejemplo de interfaz de usuario creada en Figma, que permite cumplir con la tarea “ver publicaciones de animales en adopción” del primer objetivo de la Persona Terciaria.

Figura 8. Prototipo inicial interfaz Adopciones

4. Evaluación de soluciones

Considerando el prototipo generado se realizó pruebas de usuario a 7 personas documentadas en el **ANEXO *VI***. La retroalimentación obtenida de la prueba permitió encontrar los siguientes problemas de diseño:

- Ambigüedad e inconsistencia:

    - Ambigüedad en la navegación y la identificación de funciones.
    - Diseño visual confuso o inconsistente, dificultando la comprensión de acciones.
    - Inconsistencia en el estilo y tamaño de elementos como botones, íconos y tarjetas.
- Organización y Distribución del Contenido:
    - Mala disposición espacial, como exceso de espacio en blanco o agrupación inadecuada de elementos.
    - Falta de jerarquía visual para información importante.
- Claridad en información relevante:
    - Problemas de visualización, como falta de claridad en las fotos o detalles de publicaciones.
    - Etiquetas o términos poco claros que afectan la comprensión de funciones específicas.
    - Problemas de legibilidad en textos o elementos de entrada.

Esta evaluación revela aspectos fundamentales que impactan directamente sobre la experiencia de usuario y usabilidad de manera significativa. Esta retroalimentación fue el punto de partida para mejorar el diseño en los siguientes sprints, abordando directamente los problemas identificados.

Definición de herramientas y tecnologías de desarrollo

1. Elección de tecnología para el desarrollo de la aplicación

Luego de analizar los perfiles Persona definidos en la sección anterior, se decidió que el desarrollo de la aplicación sea en una plataforma móvil (Android). Entre las seis tecnologías evaluadas para el desarrollo móvil (descritas en el **ANEXO *I***), React Native destacó por su compatibilidad en IOS y Android, su extensa documentación y el respaldo de librerías. Se seleccionó para el módulo en estudio por ofrecer una experiencia cercana al funcionamiento nativo.

2. Elección de herramienta de control de versiones

Es importante mencionar la importancia del control de cambios y versionamiento a lo largo de la implementación del presente módulo, por lo cual, se analizó tres herramientas considerando características relevantes, como se puede ver en el ***ANEXO II***, sobresaliendo Git por su arquitectura distribuida y compatibilidad con repositorios remotos como GitHub y BitBucket.

3. Elección de Editor de código

Tras un análisis de entornos integrados de desarrollo y editores de código, tomando aspectos clave para contrastarlos como se muestra en el

ANEXO III, se optó por una solución ligera y extensible como lo es Visual Studio Code.

4. Elección de Herramientas de gestión de proyectos

Después de revisar varias herramientas para gestionar proyectos y documentación en el marco de Scrum, como se muestra en el **ANEXO *IV***, se optó por Jira para la gestión de Scrum. Jira destaca por su facilidad de integración con herramientas de control de versión, compatibilidad con marcos ágiles, manejo de dependencias y capacidad para automatizar flujos. Además, como una herramienta complementaria y como solución altamente personalizable, se seleccionó Notion. Este espacio permite centralizar la documentación asociada, ofreciendo visibilidad y persistencia.

Definición de Arquitectura de software

Para el desarrollo del módulo se seleccionó el enfoque de arquitectura limpia debido a características como resistencia a cambios, definición de responsabilidades y adaptabilidad.

Planificación de SCRUM y configuración inicial de herramientas

Se seleccionó Notion como el repositorio centralizado, donde se archivaron los documentos asociados al proyecto, actividades y artefactos de Scrum (calendario de reuniones, grabaciones de daily, product backlog inicial, conclusiones de sprint retrospective). Las historias de usuario y épicas finales se registraron en la plataforma JIRA, la cual también se configuro para el manejo de 4 estados:

- Por hacer: estado inicial de todos los elementos definidos en el sprint backlog

- En curso: estado de los elementos del backlog una vez que se inicia su implementación en una rama específica de GitHub.
- Code review: fase en la que la implementación está completa y entra en revisión, seguida por la creación del Pull Request (PR).
- Listo: Se revisó y se implementó correctamente

Es importante mencionar que el identificador asignado a cada elemento no implica prioridad ni orden, sino que simplemente sirve como identificación única. Además, para mantener la sincronización se realizó la integración de JIRA con Github.

En Github se almacenó una configuración inicial del proyecto utilizando Expo y React Native, donde se definieron ramas iniciales main (rama con funcionalidades verificadas y estables) y development (rama de desarrollo donde los PRs serán integrados una vez que cumplan con el flujo definido en JIRA).

2.2 Sprint 1

2.2.1. Introducción

El objetivo del primer sprint es realizar y visualizar publicaciones de adopción, es decir, implica el desarrollo de la interfaz con el formulario de publicación y la pantalla principal para la visualización. Las decisiones adoptadas incluyen la revisión del diseño del prototipo inicial para solucionar los problemas identificados durante el DCU y la implementación de la navegabilidad principal de la aplicación móvil como un esquema base para futuras funcionalidades.

2.2.2. Planificación

En la planificación, se eligieron las historias de usuario (HU) fundamentales para implementar la sección de visualizar y crear adopciones, que se muestran en la **Tabla *2***.

Tabla 2. Historias de Usuario Sprint 1.

Identificador	**Historia de Usuario**	**Estimación**	**Tareas**
PAWQ-71	Como desarrollador, quiero diseñar los mockups del proyecto en Figma para visualizar y comunicar eficazmente la interfaz de usuario a los miembros del equipo y a los interesados.	21	• Refinamiento de prototipo inicial en figma

PAWQ-28	Como persona interesada en adoptar, quiero ver las publicaciones de animales en adopción para encontrar un animal de mi interés al cual pueda brindarle un hogar.	89	• Crear interfaz "Adopciones" en donde se muestren las publicaciones de animales disponibles para adopción. • Creación de un sistema de filtro para publicaciones, por fecha, ubicación y especie. • Crear sistema de navegación • Implementación de lógica de navegación • Definición de Repository y entidades de dominio
PAWQ-27	Como persona interesada en dar en adopción animales de compañía, quiero realizar publicaciones con información específica del animal para informar a los adoptantes sobre el perfil del animal que deseo dar en adopción	55	• Crear interfaz "Agregar nueva publicación de Adopción" con el formulario de características del animal en adopción. • Crear el botón "Subir foto”. • Crear el botón "Nueva adopción" para redactar una nueva publicación de adopción. • Creación del botón de "publicar".

2.2.3. Implementación

Previo a cualquier implementación en código, se realizó varias modificaciones en el prototipo inicial, en las interfaces de presentación de publicaciones de adopción. En concordancia con la arquitectura en capas establecida antes, se realizaron las abstracciones y definiciones de entidades pertinentes, alineadas con el objetivo del sprint. En este proceso, se crearon interfaces TypeScript que representan las entidades provenientes del backend, se implementaron hooks de React para abordar los casos de uso y se diseñaron componentes funcionales de React para la navegación y la interfaz de usuario como se muestra en la Figura 9.

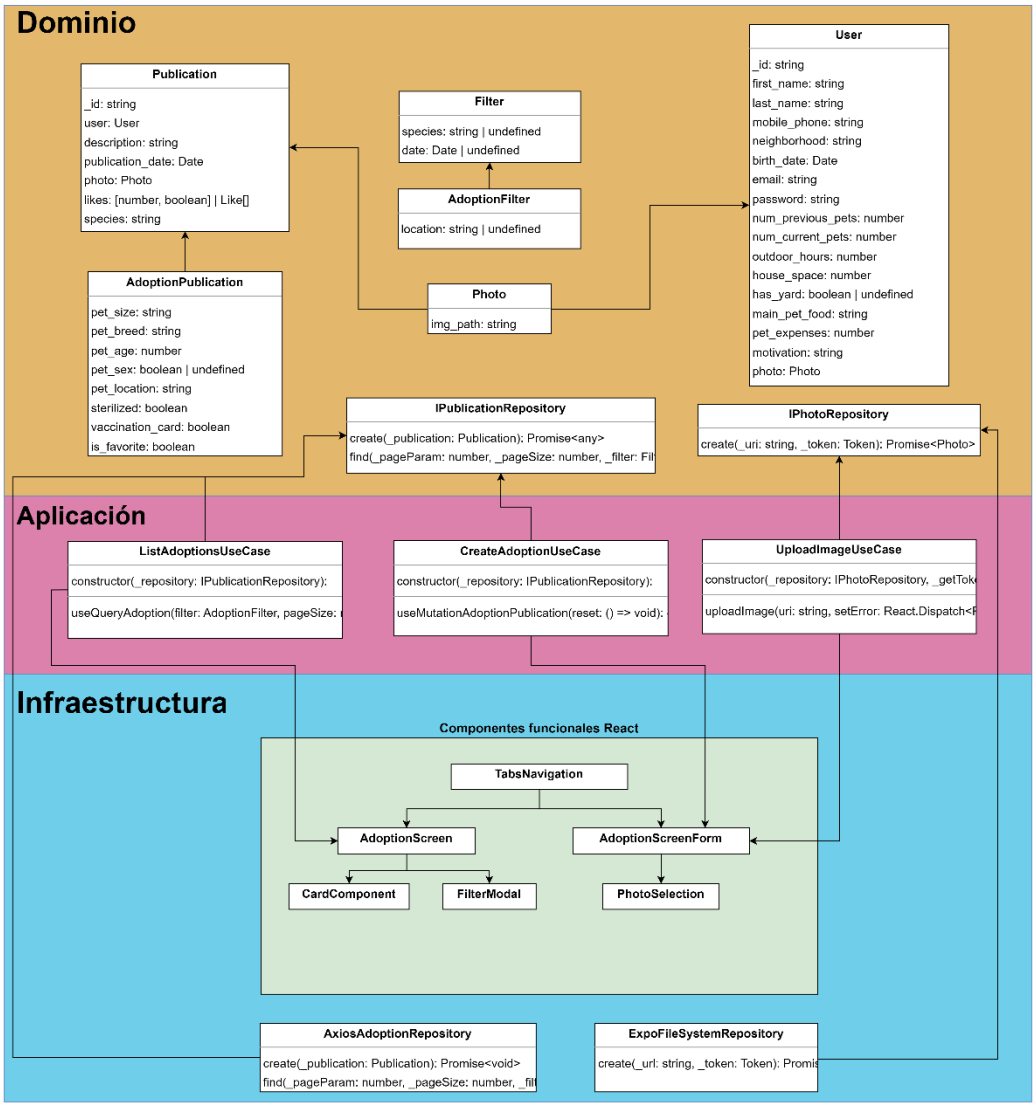
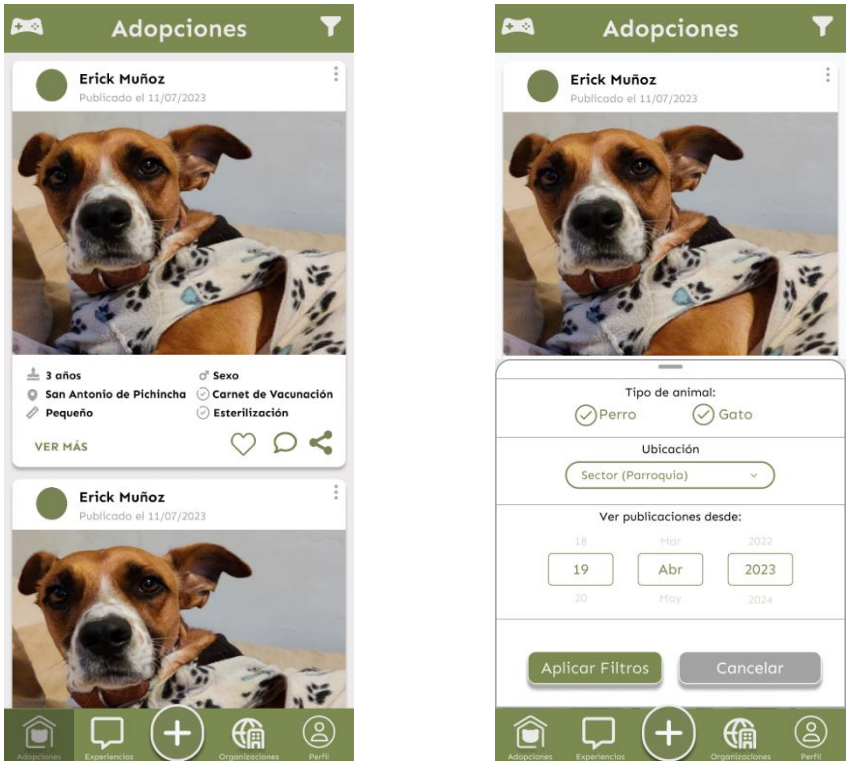


Figura 9. Arquitectura limpia para Sprint 1

A continuación, se detalla el enfoque utilizado para cada historia de usuario considerando las tareas relacionadas y los elementos mencionados en la **Figura *9***:

- PAWQ-71 Diseño de interfaces de usuario: Tomando el prototipo inicial resultado de la fase de generación de soluciones del DCU, se seleccionaron las interfaces para mostrar las publicaciones de adopciones y se realizaron mejoras considerando los problemas encontrados en la fase de evaluación como se muestra en los diseños realizados en Figma de la **Figura *10***.



(a) (b)

Figura 10. Diseño refinado de la interfaz adopciones (a) y filtro de adopciones (b)

Tras las modificaciones en el prototipo, se han identificado 6 componentes React principales que son parte de la arquitectura definida en la Figura 9:

- TabsNavigation: Encargado del sistema de navegación y las pantallas en la aplicación.
- AdoptionScreen: Dedicado a las publicaciones de adopciones
- FilterModal: Modal para visualizar los filtros en la pantalla de adopciones
- CardComponent: Representación de una publicación de adopción en una tarjeta.
- AdoptionScreenForm: Formulario para la creación de publicaciones de adopción.
- PhotoSelection: Encargado de cargar la foto desde la galería

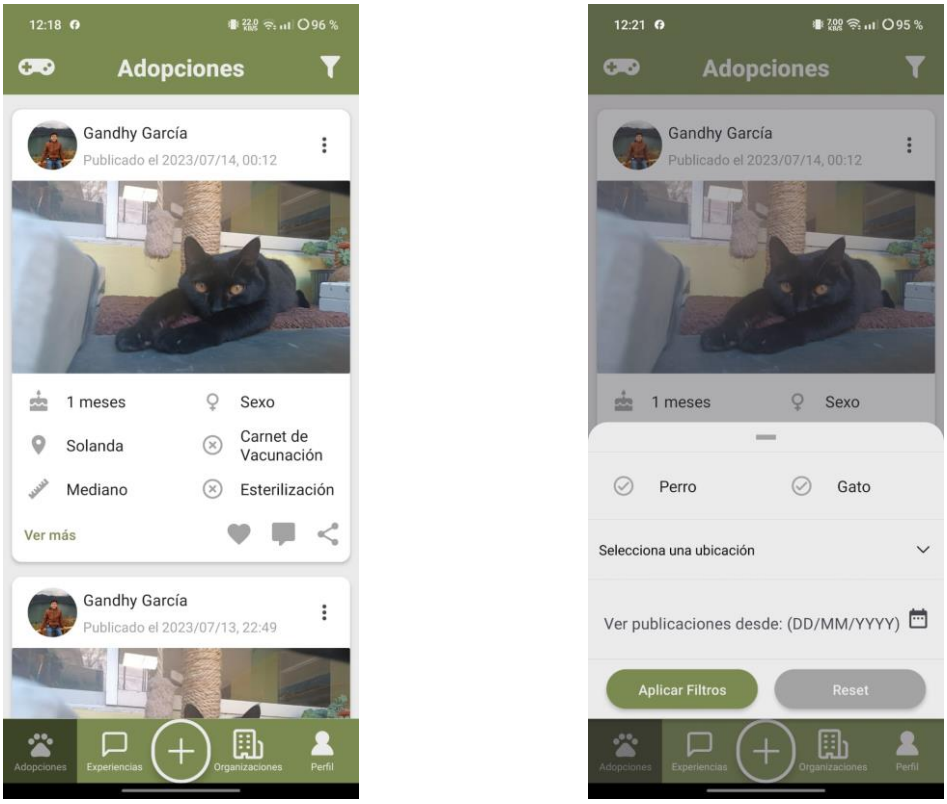
- PAWQ-28 Ver publicaciones de adopciones: Considerando la arquitectura definida, se comienza por el componente AdoptionScreen, situado en la capa de infraestructura, el cual emplea el hook useQueryAdoption de la clase

ListAdoptionUseCase, como se ilustra en la **Figura *9***. Este hook, en la capa de aplicación, actúa como un caso de uso que se apoya en la interfaz IPublicationRepository de la capa de dominio. Dicha interfaz define la estructura para interactuar con los datos de publicaciones. Mientras que la implementación concreta AxiosAdoptionRepository, localizada en la capa de infraestructura, establece la manera de acceder y manipular realmente la información desde y hacia la fuente de datos externa, en este caso, la API backend. Para mantener la coherencia con las reglas de dependencia en la arquitectura limpia, se emplea la técnica de inversión de control mediante la inyección de dependencias. Esta técnica asegura que la implementación de la interfaz repository permanezca en la capa de infraestructura y se cumpla con las reglas de dependencia de una arquitectura limpia.

- PAWQ-27 Publicar información específica del animal: Para implementar esta funcionalidad se introdujeron dos nuevos casos de uso, UploadImageUseCase y CreateAdoptionUseCase, junto con una nueva interfaz, IPhotoRepository, manteniendo la arquitectura existente. El flujo de información comienza en el componente AdoptionScreenForm, el cual emplea los hooks de los casos de uso mencionados y se apoya en las abstracciones de IPublicationRepository y la nueva interfaz IPhotoRepository. La implementación específica de esta última, ExpoFileSystemPhotoRepository, define el proceso de envío de imágenes al servicio backend.

2.2.4. Revisión

Considerando la implementación en la aplicación móvil en la sección anterior, en **Figura *11*** y **Figura *12*** se muestra el resultado de las interfaces implementadas en el sprint 1.



(a) (b)

Figura 11. Interfaces en el Sprint 1: adopciones (a) y filtro de adopciones (b)

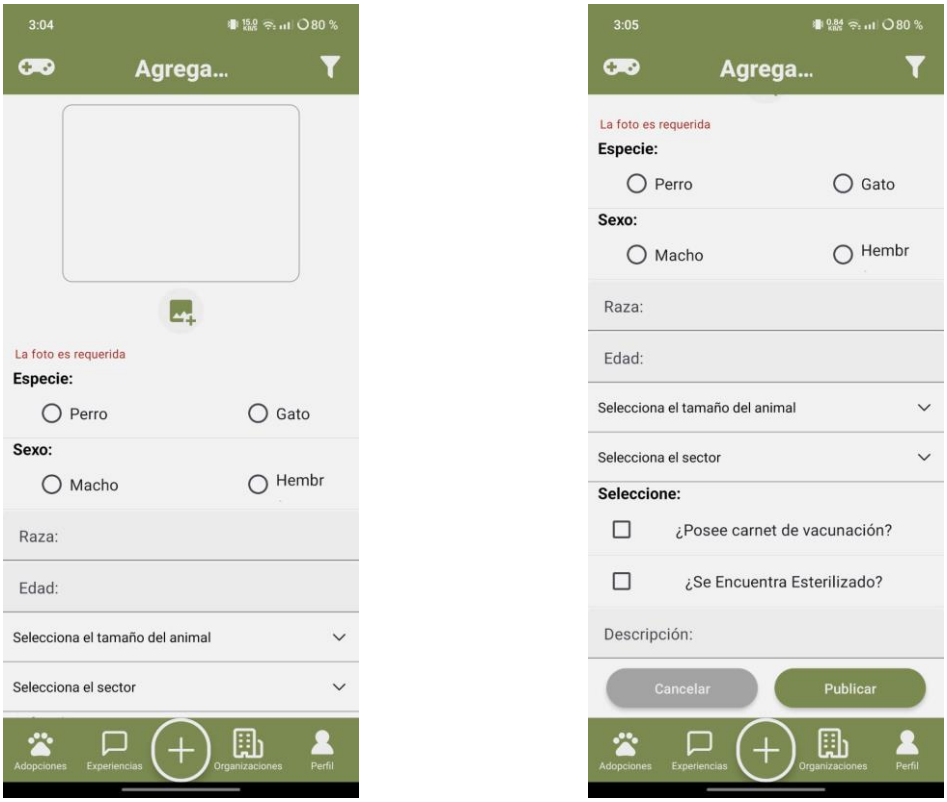


Figura 12. Interfaz en sprint 1: formulario para agregar publicación de adopción

En el desarrollo de este sprint, se evidenció una discrepancia en la estimación y el esfuerzo real destinados a la historia de usuario PAWQ-27 como se muestra en la **Tabla *3***, lo cual causó que el resultado final no cumpla con la definición de Done.

Tabla 3. Tabla comparativa de esfuerzo sprint 1

Historia de usuario	Esfuerzo estimado	Esfuerzo efectivo
PAWQ-71	21	21
PAWQ-28	89	89
PAWQ-27	89	144

2.2.5. Retrospectiva

Durante la reunión retrospectiva, se realizaron varias conclusiones e identificaron acciones de mejora para el éxito y la eficiencia en los próximos sprint:

1. Mejorar la estimación de las historias de usuario aplicando valores más exactos a pesar de la escala y considerando el tiempo necesario para pruebas.
2. Realizar una revisión de la definición de Done antes de iniciar cada sprint.
3. Mantener una comunicación efectiva dentro del equipo. Respecto a esto, se modificó el registro de documentación para las reuniones Daily, así como la continuidad en la revisión de código y el flujo de trabajo establecido.

2.3 Sprint 2

2.3.1. Introducción

El segundo sprint se centra en permitir al usuario publicar y visualizar experiencias, así como guardar publicaciones de adopción como favoritas. A la pila de trabajo de este sprint se incluyen: a) las tareas no completadas en el sprint anterior, b) ajuste de la navegación para las nuevas interfaces maximizando la reutilización de componentes, c) revisión de Done antes de iniciar y d) mantener la identidad visual y los estándares de diseño establecidos en el primer sprint.

2.3.2. Planificación

En la planificación, se consideraron las historias de usuario (HU) (como se muestra en la **Tabla *4***) para las publicaciones de experiencias junto con las HU no completada en el sprint anterior (PAWQ-27 descrita en la **Tabla *1***).

Tabla 4. Historias de Usuario Sprint 2.

Identificador	Historia de Usuario	Estimación	Tareas
PAWQ-29	Como usuario interesado en adoptar animales de compañía, quiero poder guardar publicaciones de animales en adopción como favoritos para acceder fácilmente a las opciones que más me interesan en el futuro.	34	• Agregar el botón "Favoritos" en cada publicación en la pantalla de "Adopciones". • Crear la interfaz "Favoritos" para visualizar los favoritos del adoptante.
PAWQ-27	Como persona interesada en dar en adopción animales de compañía, quiero realizar publicaciones con información específica del animal para informar a los adoptantes sobre el perfil del animal que deseo dar en adopción.	55	• Crear interfaz "Agregar nueva publicación de Adopción" con el formulario de características del animal en adopción. • Crear el botón "Subir foto". • Crear el botón "Nueva adopción" para redactar una nueva publicación de adopción. • Creación del botón de "publicar".
PAWQ-12	Como promotor de la adopción responsable, quiero crear publicaciones sobre mi experiencia al adoptar animales para motivar a otras personas a adoptar animales de compañía.	30	• Crear la interfaz "Agregar una nueva publicación de experiencia" con el formulario y los atributos pertinentes. • Agregar botón publicar en la interfaz "Agregar una nueva publicación de experiencia". • Crear el botón "nueva experiencia" para crear una publicación de experiencia. • Reutilizar el componente de "Subir foto" adaptándolo a la interfaz "Agregar una nueva publicación de experiencia".
PAWQ-15	Como persona que promueve la adopción responsable, quiero poder ver las publicaciones con experiencias de otros usuarios para sentirme inspirado por las historias exitosas de adopción y ver cómo los animales han encontrado un hogar.	21	• Crear el sistema de filtro para las publicaciones de experiencias por fecha y especie. • Crear la interfaz "Experiencias" en donde se visualicen todas las publicaciones de experiencias. • Implementar lógica de paginación en la interfaz "Experiencias".

2.3.3. Implementación

En concordancia con la arquitectura, se mantuvo las abstracciones y definiciones de entidades previas y se agregaron nuevas, consistentes con la sección de “Experiencias” y “Favoritos”.

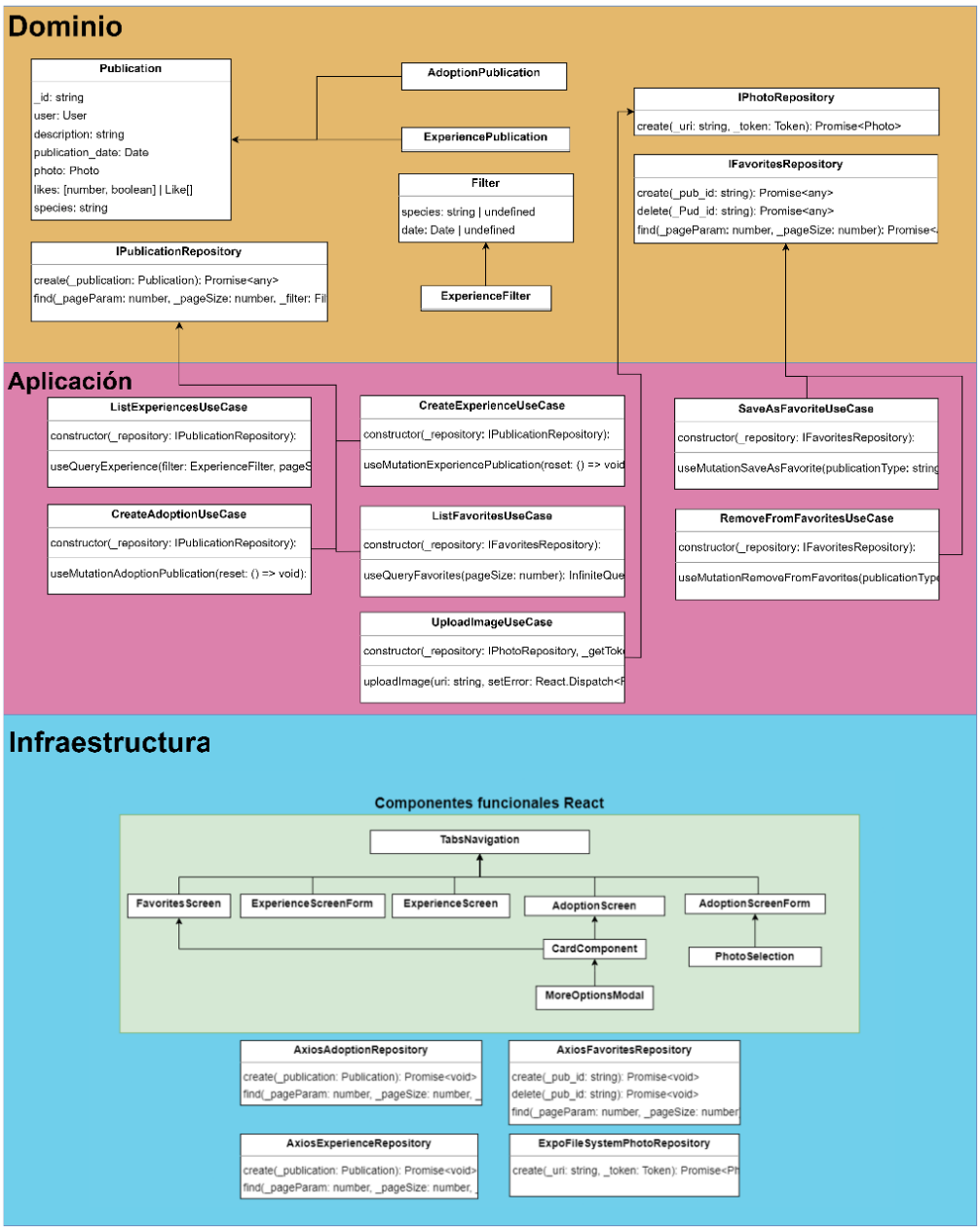


Figura 13. Arquitectura limpia para Sprint 2

De igual forma que en el sprint previo, se implementaron hooks de React para los casos de uso y se diseñaron componentes funcionales de React y las nuevas interfaces de usuario como se muestra en la Figura 13.

A continuación, se detalla el enfoque utilizado para cada historia de usuario considerando las tareas relacionadas y los elementos mencionados en la **Figura *13***:

- PAWQ-29 Guardar publicaciones de adopción como favoritos: Para mantener la separación de responsabilidades en las capas, se ha introducido una nueva interfaz Repository dedicada al manejo de favoritos en el dominio como se muestra en la **Figura *13***. A partir de esta interfaz, se han desarrollado los casos de uso SaveAsFavoriteUseCase y RemoveFromFavoritesUseCase para agregar y eliminar favoritos, respectivamente. Estos casos de uso se integran en las interfaces FavoritesScreen y AdoptionScreen, donde se incluye el componente MoreOptionModal para cada publicación, ofreciendo la opción de guardarlas como favoritas. Asimismo, se ha implementado AxiosExperienceRepository para administrar la conexión con el backend y que se encarga de gestionar las publicaciones de experiencias. Este enfoque aplica la técnica de inversión de control.

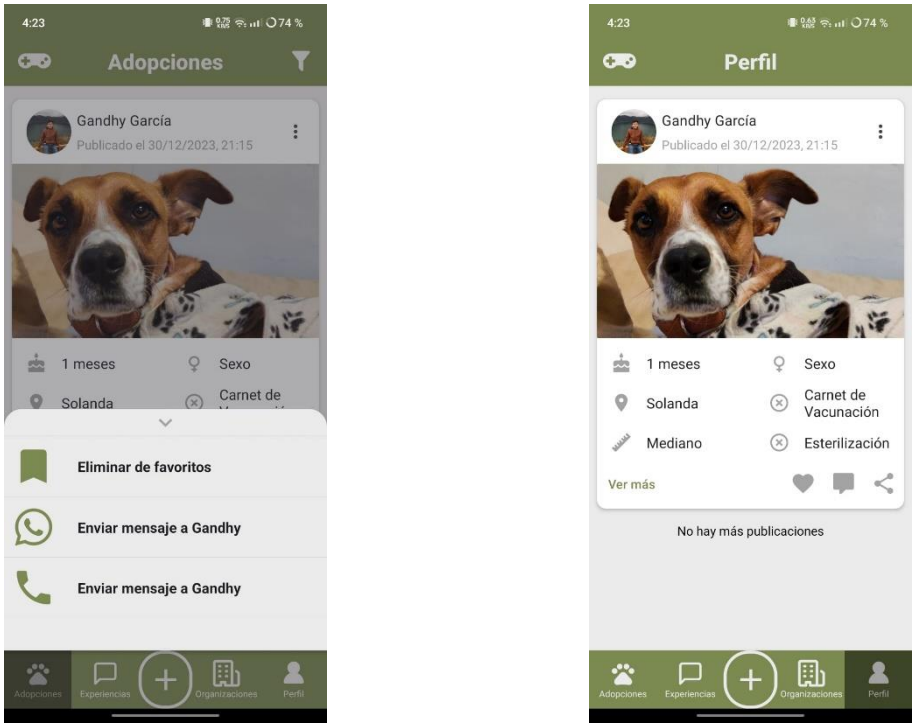
- PAWQ-27 Publicar información específica del animal: Para terminar con el desarrollo de esta HU, se realizó una nueva revisión de código y pruebas para la detección y solución de errores y bugs, manteniendo la arquitectura de software como se muestra en la **Figura *13***.

- PAWQ-12 Crear publicaciones de experiencias: Basándose en el desarrollo de publicaciones de adopciones, se replicó la misma estructura de capas para la creación de publicaciones de experiencias. Se introdujeron nuevas entidades como publicaciones de experiencia (ExperiencePublication) en el dominio, junto con el caso de uso CreateExperienceUseCase, que se muestra en la **Figura *13***. Se mantuvo el uso de la interfaz IPublicationRepository y se implementó una nueva versión de esta en la capa de infraestructura, aplicando la técnica de inversión de control. Además, en esta misma capa, se diseñó la interfaz de usuario ExperienceScreenForm, que emplea el caso de uso CreateExperienceUseCase.

- PAWQ-15 Visualizar publicaciones de experiencias: Para la interfaz de usuario adopciones, se replicó la estructura de la arquitectura en capas, utilizando las nuevas entidades en el dominio y creando el caso de uso ListExperiencesUseCase, que se muestra en la **Figura *13***. Este caso de uso se apoya en la interfaz IPublicationRepository y se emplea en la interfaz de usuario ExperienceScreen.

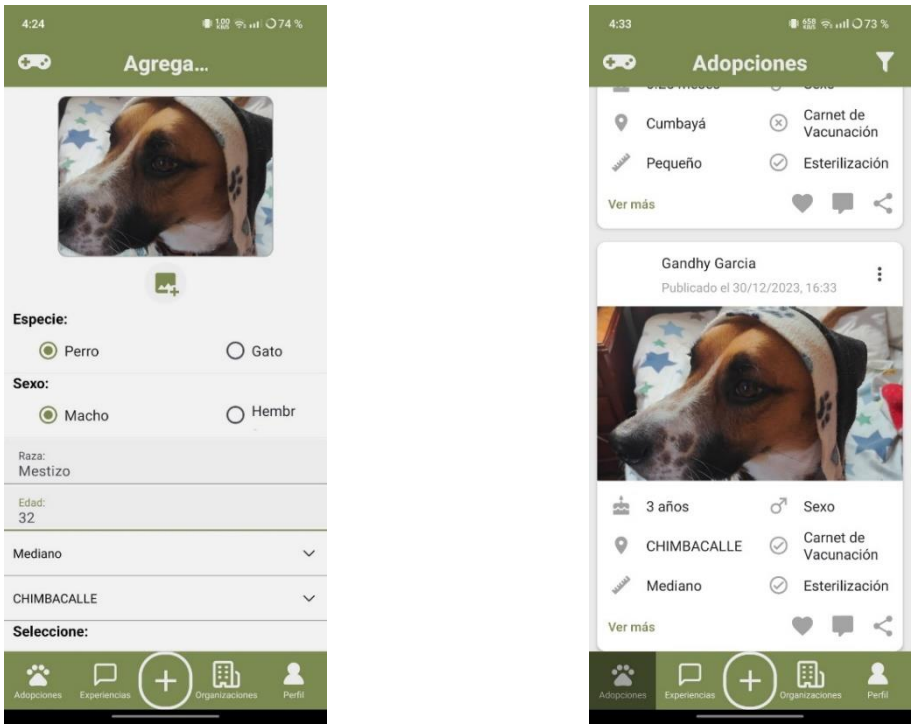
2.3.4. Revisión

Considerando la HU no completada en el sprint anterior, se muestra la implementación de las HU en **Figura *14***, **Figura *15***, **Figura *16*** del sprint 2.



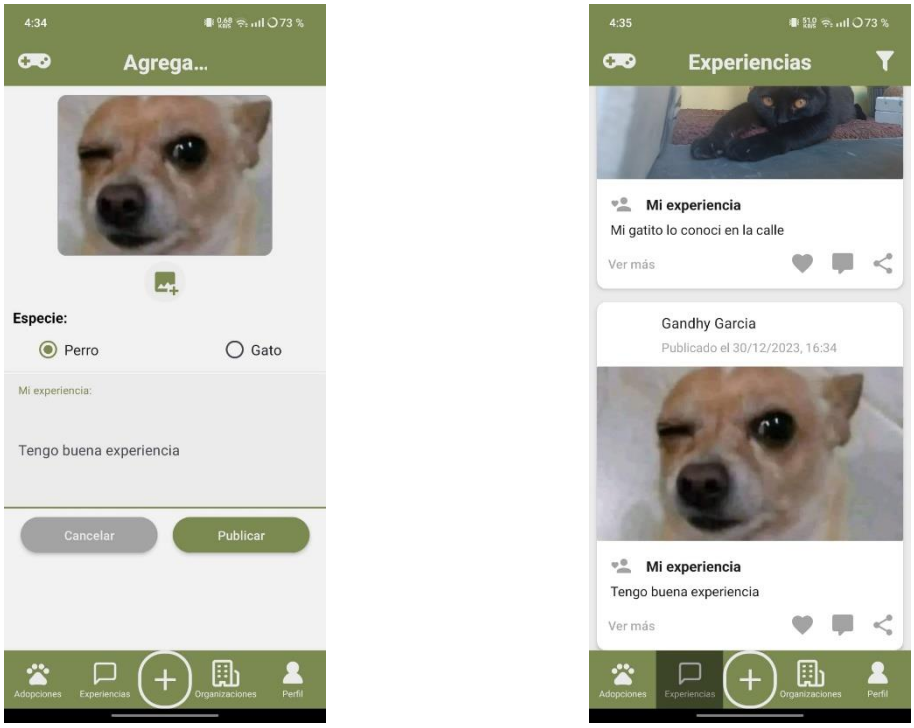
(a) (b)

Figura 14. Interfaces en Sprint 2: modal para marcar como favorito (a) y favoritos (b)



(a) (b)

Figura 15. Interfaz en Sprint 2: formulario para agregar publicación (a) y publicación realizada (b)



(a) (b)

Figura 16. Interfaces en Sprint 2: formulario para agregar nueva experiencia (a) y publicaciones de experiencias (b)

Durante este sprint, se observó una mejora en la precisión de las estimaciones de esfuerzo para cada historia de usuario, como se muestra en la **Tabla *5***. Esto permitió cumplir con los criterios definidos para el Done, lo que resultó en un incremento funcional, sin embargo, se definieron cambios a nivel de diseño de interfaz de usuario para el siguiente sprint.

Tabla 5. Tabla comparativa de esfuerzo sprint 2

Historia de usuario	Esfuerzo estimado	Esfuerzo efectivo
PAWQ-29	34	34
PAWQ-27	55	55
PAWQ-12	30	30
PAWQ-15	21	21

2.3.5. Retrospectiva

En la retrospectiva del segundo sprint se identificaron diferentes áreas de mejora con el apoyo de una dinámica de retrospectiva denominada “matriz de aprendizaje” que consiste en una matriz de cuatro cuadrantes (¿Qué nos ha gustado?, ¿Qué no nos ha gustado?, Nuevas ideas, A tener en cuenta), como se muestra en la **Figura *17***. Los resultados de la ejecución de la dinámica derivaron las siguientes decisiones:

1. Mantener prácticas clave como las revisiones diarias, la comunicación fluida y el seguimiento del código.
2. Tomar acciones para mejorar como solicitar ayuda a tiempo, comunicar retrasos, evitar commits masivos en Git, revisar el código de referencia, afinar el estándar de codificación, realizar revisiones conjuntas y pruebas exhaustivas más temprano.
3. Aumentar la proactividad para aprender las librerías necesarias, abordar bloqueos prolongados mediante la programación grupal.
4. Reasignar responsabilidades en casos extraordinarios.

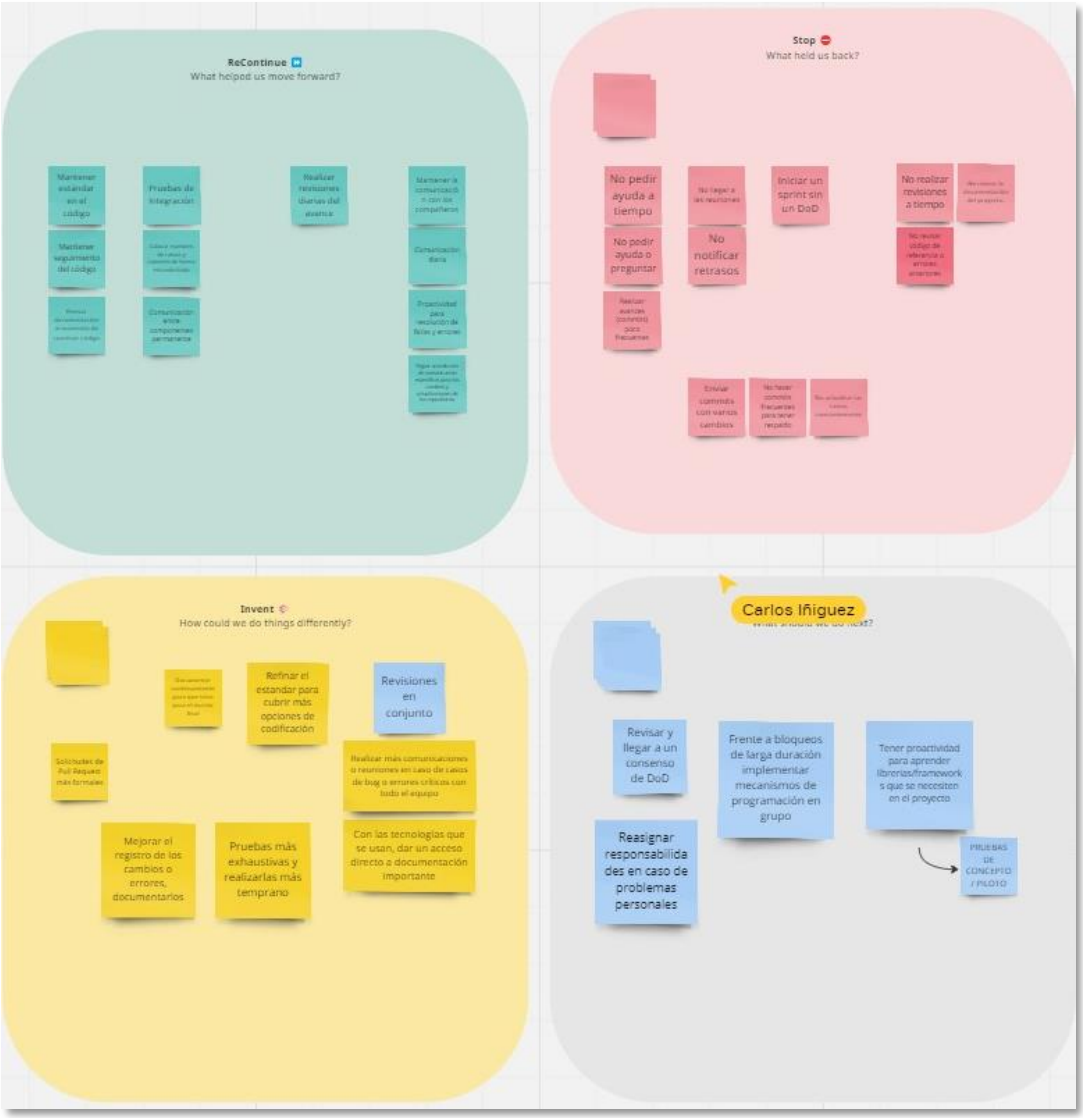


Figura 17. Ejemplo de tablero sprint retrospective, sprint 2

2.4 Sprint 3

2.4.1. Introducción

El objetivo del tercer sprint es identificar personas aptas para la adopción mediante el registro y visualización de sus características. En este sprint se tomaron decisiones para abordar los cambios solicitados en la revisión anterior: a) reestructurar la navegación de la aplicación para mejorar la escalabilidad, especialmente en las secciones de perfil y favoritos, y b) modificar el componente TabsNavigation (que incluye la barra de navegación y encabezado de cada pantalla) para ajustarse a los cambios.

2.4.2. Planificación

Tomando como base el objetivo del sprint 3 y los cambios solicitados se escogieron 5 product backlog items (PBI): 2 historias de usuario (PAWQ-26 y PAWQ-25) y 3 problemas de diseño o errores (PAWQ-120, PAWQ-121 y PAWQ-122), especificados en la **Tabla *6***.

Tabla 6. Product Backlog Items sprint 3.

Identificador	**Product backlog item**	**Estimación**	**Tareas**
PAWQ-26	Como persona interesada en adoptar animales de compañía, quiero registrar información sobre mi capacidad de adopción para que la persona que pone en adopción sepa que la mascota tendrá un cuidado responsable bajo mi protección.	25	• Creación de la interfaz "Formulario de Aptitud".
PAWQ-25	Como persona interesada en dar en adopción animales de compañía, quiero identificar a las personas aptas para una adopción responsable.	25	• Crear interfaz "Perfil de usuario" • Crear sección de "Aptitud" en la interfaz de "Perfil de usuario" • Crear sección de comentarios para cada publicación • Crear pantalla "Mis publicaciones"
PAWQ-120	Reubicar el icono de juego	34	• Añadir el icono de juego en la pestaña de navegación o alinear el título a la izquierda y ubicar ambos íconos en la sección derecha.

PAWQ-122	Visibilidad de publicaciones de favoritos	21	• Agregar un ícono en la sección de acciones de la tarjeta (publicaciones de adopciones), integrar el botón de favoritos y mostrar un pop-up/snackbar para confirmar la selección como favorito.
PAWQ-121	Rediseño de botón "añadir publicación"	5	• Reducir el tamaño del botón y centrarlo, cambiar la paleta de colores y hacerlo completamente circular.

2.4.3. Implementación

En este sprint se identificaron los elementos de dominio necesarios para almacenar la información de aptitud de cada persona. Se optó por modificar un subconjunto de los datos definidos previamente. Se implementó un hook para facilitar esta modificación y se llevó a cabo una refactorización del componente TabsNavigation, ahora renombrado como SocialNavigator. Por último, se desarrollaron los cambios de interfaz definidos en la **Tabla *6***.

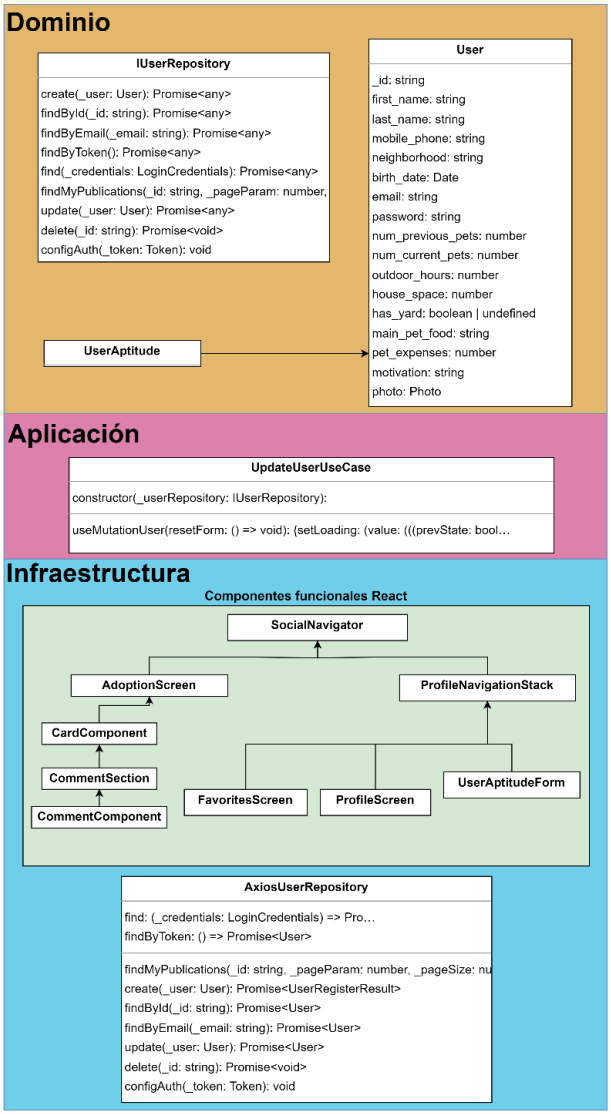


Figura 18. Arquitectura limpia para Sprint 3

A continuación, se detalla el enfoque utilizado en cada PBI considerando las tareas relacionadas y los elementos mencionados en la **Figura *18***:

- PAWQ-26 Registrar aptitud para adoptar: Se llevó a cabo el registro de aptitud mediante la identificación de aspectos clave en la entidad User, derivando en la creación de la entidad UserAptitude y la interfaz IUserRepository. Además, se definió la clase UpdateUserCase junto con su hook correspondiente. Se completó la construcción del componente ProfileScreen para modificar la información de la sección de aptitud. Finalmente, para gestionar los datos con la API backend, se implementó AxiosUserRepository.

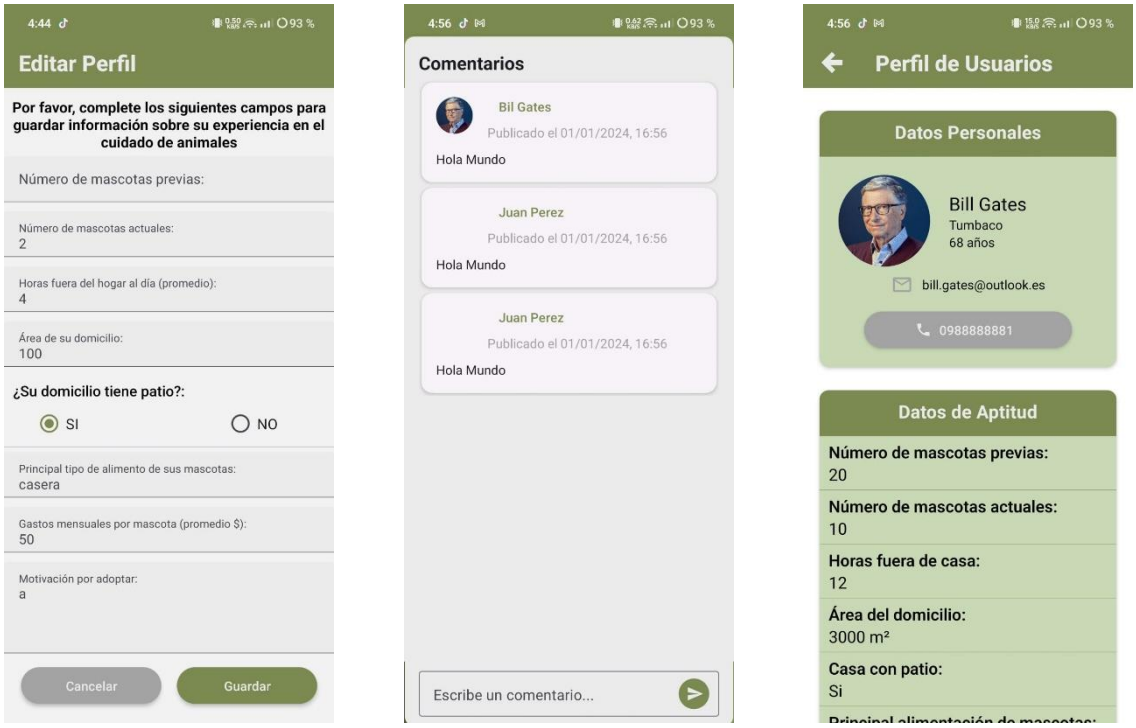
- PAWQ-25 Identificar personas aptas para adopción: Se desarrollaron los componentes CommentSection y CommentComponent para visualizar perfiles de

posibles adoptantes. En este sprint, estos componentes muestran usuarios de ejemplo, dado que la funcionalidad real no está implementada. De esta forma, es posible acceder a la pantalla ProfileScreen de cada posible adoptante para ver sus datos de aptitud.

- PAWQ-120 Reubicar icono de juego: Se llevó a cabo una modificación en los iconos superiores mediante un refactor del componente SocialNavigator. Se creó un encabezado personalizado donde los iconos fueron reubicados a la derecha y el texto se colocó a la izquierda.
- PAWQ-122 Visibilidad de publicaciones de favoritos: Se realizó una modificación en el componente CardComponent para mejorar la visibilidad al marcar o desmarcar como favorito una publicación. Ahora, al realizar estas acciones, se muestra un mensaje tipo SnackBar para visibilizar la acción.
- PAWQ-121 Rediseño de botón de añadir publicaciones: Se abordó el problema de diseño mediante un refactor del componente SocialNavigator y una actualización en el diseño del botón central de la barra de navegación.

2.4.4. Revisión

Durante la revisión, se verificó implementación de las HUs y PBIs listados en la **Tabla *6***. Los resultados se presentan a continuación: la **Figura *19*** muestra las interfaces relacionadas con el registro de la capacidad de adopción y su visualización. En la **¡Error! No se encuentra el origen de la referencia.** se muestran los cambios de diseño implementados en los botones, y en la **Figura *20*** se aborda la visibilidad del marcado como favorito.



(a) (b) (c)

Figura 19. Interfaces en Sprint 3: registro de aptitud (a), sección de comentarios (b) y aptitud de adoptantes (c)

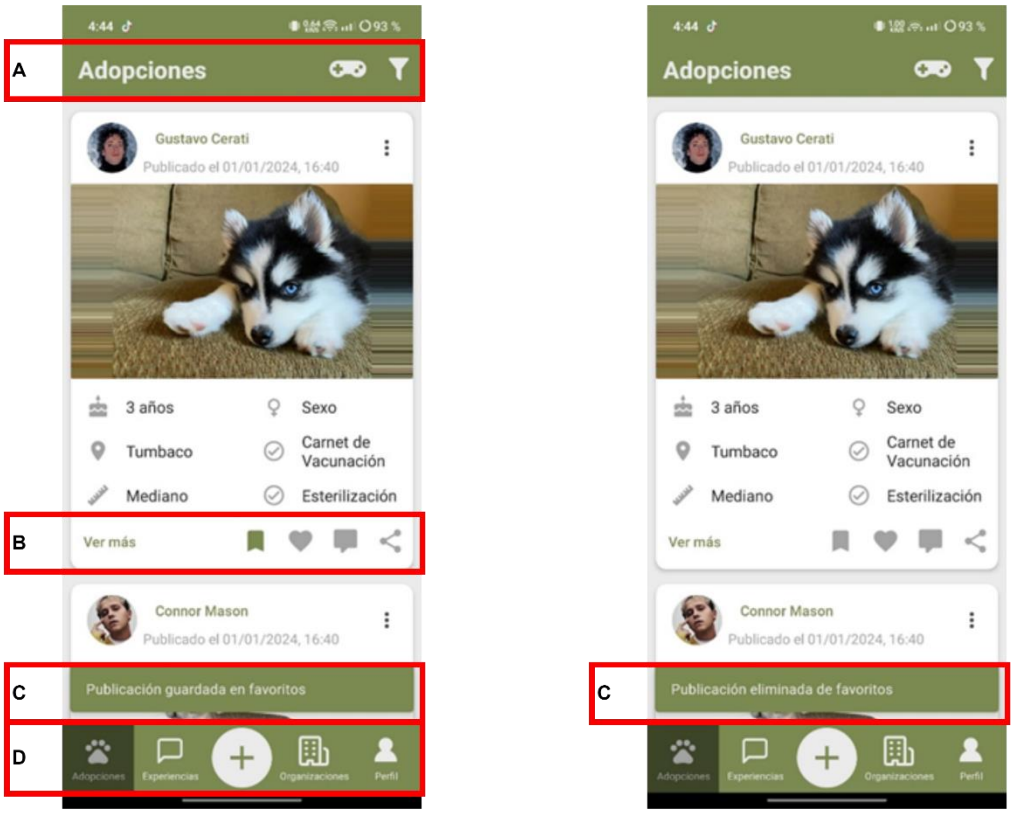


Figura 20. Interfaz en Sprint 3: cambio de botones en A, B y C y visibilidad de favoritos en D

Durante el sprint 3, se observó una diferencia entre el esfuerzo estimado y el esfuerzo real empleado, como se muestra en la **Tabla *7*** . Sin embargo, se logró aprovechar el esfuerzo excedente para compensar el faltante. Esto permitió producir un incremento funcional, no obstante, se identificó un cambio de diseño en la sección de perfil de usuario que generó un nuevo PBI para su ejecución en posteriores sprints.

Tabla 7. Tabla comparativa de esfuerzo sprint 3

Historia de usuario	Esfuerzo estimado	Esfuerzo efectivo
PAWQ-26	25	25
PAWQ-25	25	25
PAWQ-120	34	32
PAWQ-122	21	25
PAWQ-121	5	4

2.4.5. Retrospectiva

Durante la reunión de retrospectiva, se reconoció la efectividad de las acciones tomadas para mejorar el rendimiento en los sprints anteriores, por lo que se decidió mantenerlas y mejorarlas.

2.5 Sprint 4

2.5.1. Introducción

El objetivo del sprint 4 es habilitar la funcionalidad de likes, comentarios y la interacción con otros usuarios a través de aplicaciones externas. Se busca también la capacidad de compartir publicaciones con usuarios que no pertenezcan a la aplicación. Para lograrlo, se seleccionó una herramienta para facilitar la interacción con la aplicación externa WhatsApp. Por otro lado, se abordó un cambio de diseño de la sección de perfil de usuario, mencionado en el sprint anterior.

2.5.2. Planificación

En este sprint, se seleccionaron los PBIs relacionados con la interacción del usuario en cada publicación como se muestra en la **Tabla *8***, estimando cada una en función de las acciones necesarias para las funcionalidades en las Historias de Usuario y los cambios requeridos.

Tabla 8. Product Backlog Items sprint 4.

Identificador	Product backlog item	Estimación	Tareas
PAWQ-14	Como promotor de la adopción responsable, quiero hacer likes y agregar comentarios para mostrar mi interés y expresar mi opinión.	55	• Crear el botón de "like" en la sección de “Interacciones Sociales” • Crear el botón de “comentarios” en la sección de “Interacciones Sociales”
PAWQ-13	Como promotor de la adopción responsable, quiero poder compartir publicaciones de animales de compañía en adopción con personas fuera de la aplicación para ampliar el alcance de posibles adoptantes.	55	• De cada publicación de adopciones crear botón “Compartir” en la sección de “Interacciones Sociales”. • Incorporar API para generar y compartir una imagen de la publicación. • Prueba de concepto de API's para generar imagen y para compartir en Whatsapp.
PAWQ-30	Como usuario interesado en adoptar animales de compañía, quiero poder contactarme con las personas que realizan las publicaciones de adopción a través del número celular para aclarar dudas y coordinar detalles de la entrega del animal.	21	• Crear un botón que redireccione a WhatsApp con el número de contacto de la persona que da en adopción. • Crear un botón que redireccione a una llamada con el número de contacto de la persona que da en adopción. • Prueba de concepto de API para abrir aplicación nativa de llamada.
PAWQ-128	Diseño de pantalla perfil de usuario	5	• Definir un tamaño de letra y quitar el bold en la fuente de la sección de Aptitud. • Cambiar el verde claro del fondo de los datos (más claro).

2.5.3. Implementación

Durante este sprint, se definieron nuevas entidades esenciales para gestionar interacciones. Se desarrollaron casos de uso para ejecutar estas acciones por medio del

uso de hooks. Además, se llevaron a cabo modificaciones en varios componentes para integrar nuevas funcionalidades y emplear los nuevos casos de uso mostrados en la **Figura *21***.

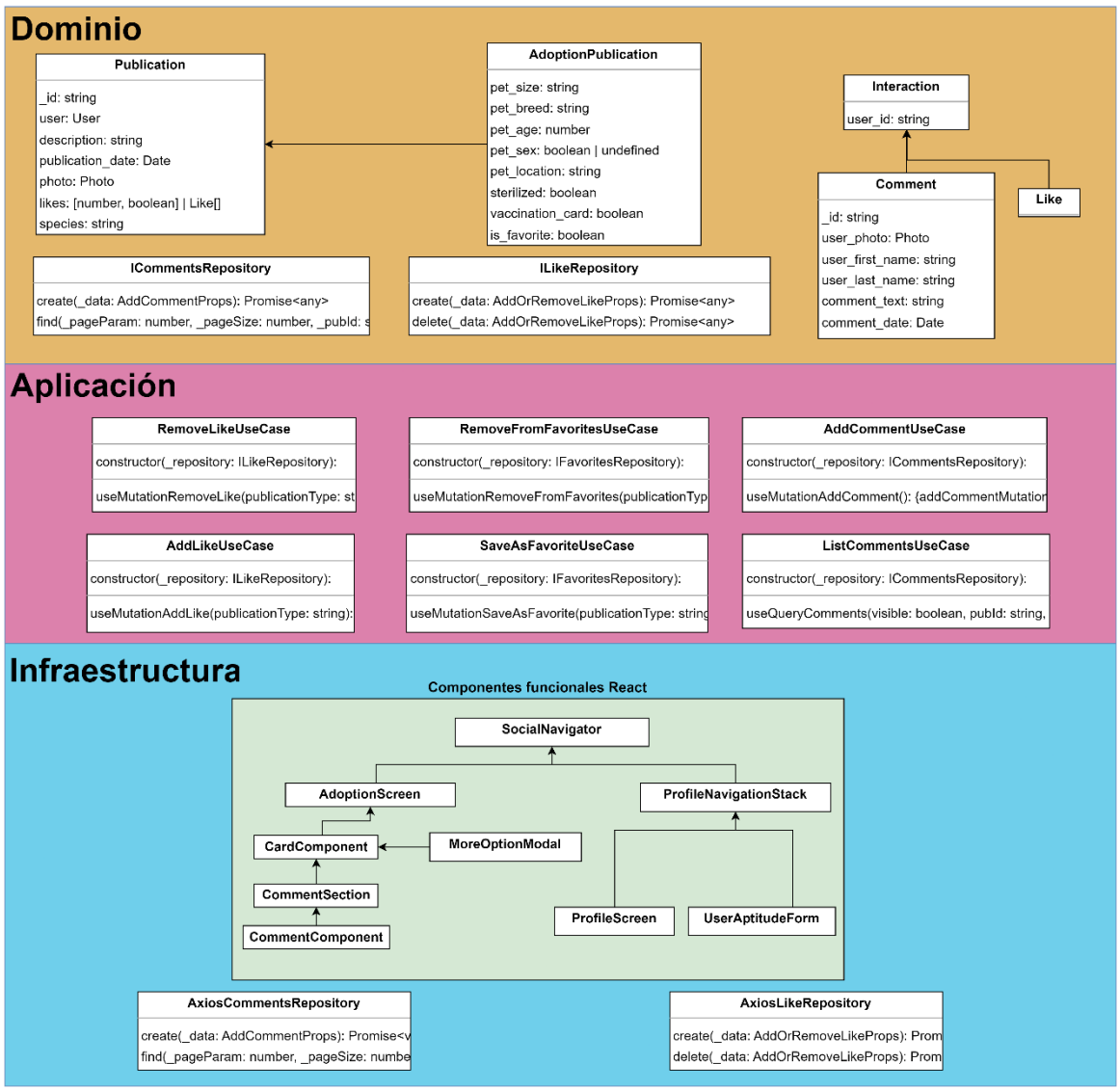


Figura 21. Arquitectura limpia para sprint 4

A continuación, se detalla el enfoque utilizado para cada PBI considerando las tareas relacionadas y los elementos mencionados en la **Figura *21***:

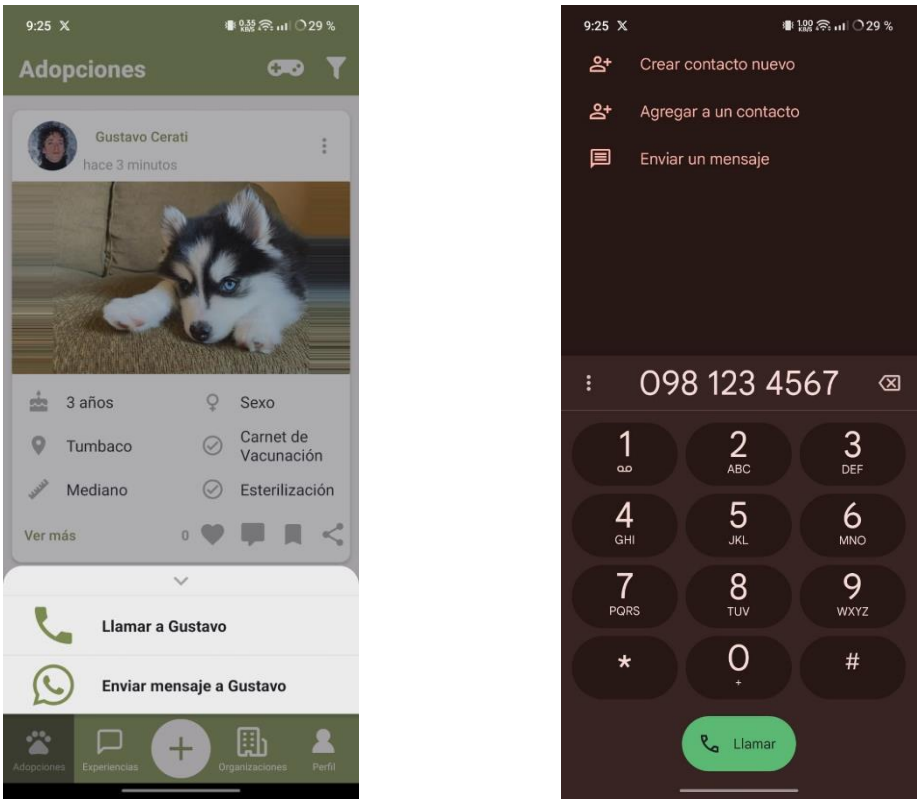
- PAWQ-14 Realizar likes y comentarios: Se establecieron las interfaces ICommentsRepository y ILikeRepository siguiendo la estructura de la arquitectura limpia. Además, se definieron entidades de dominio como Interaction, Comment y Like. Se crearon casos de uso específicos: AddLikeUseCase y RemoveLikeUseCase para agregar y eliminar likes, así como AddCommentUseCase y ListCommentsUseCase para agregar y listar comentarios

respectivamente. Estos casos de uso se emplean en la interfaz de AdoptionScreen, en cada publicación (CardComponent).

- PAWQ-13 Compartir publicaciones fuera de la aplicación: Se implementó esta funcionalidad utilizando Expo SDK que facilitó la generación de una imagen a partir del componente CardComponent, el cual representa cada publicación de adopción.
- PAWQ-30 Contactarme con adoptantes para confirmar entrega: Utilizando los datos del componente CardComponent, se empleó Expo SDK para enviar el número registrado en el perfil directamente a la aplicación nativa para realizar llamadas en la plataforma móvil.
- PAWQ-128 Diseño de pantalla perfil de usuario: El problema de diseño en la interfaz del perfil de usuario se solventó al modificar el tipo de letra y el color.

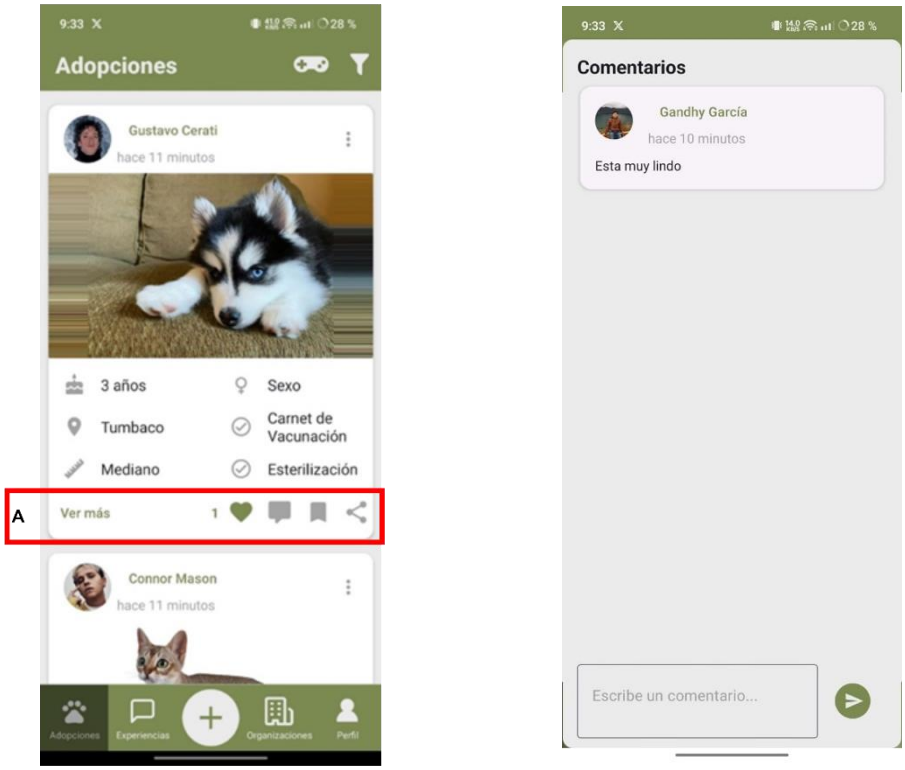
2.5.4. Revisión

Durante esta revisión, se visualizaron las interfaces y cambios realizados en la sección anterior, como se muestra en **Figura *22***, **Figura *23*** y **Figura *24***.



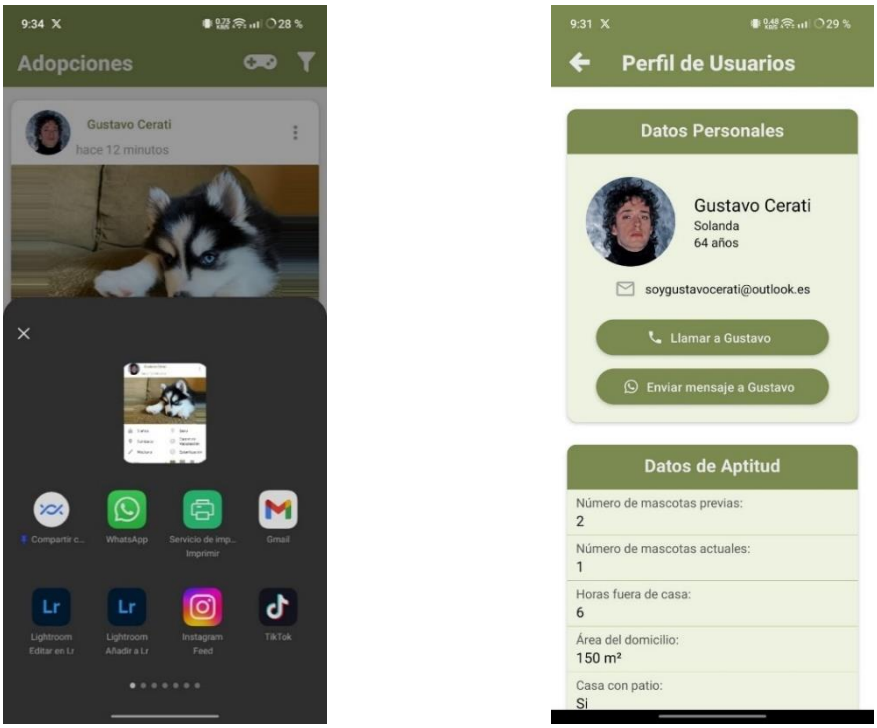
(a) (b)

Figura 22. Interfaces en Sprint 4: Modal para llamadas (a) y aplicación de llamada con el número (b)



(a) (b)

Figura 23. Interfaces en Sprint 4: like en adopciones en A (a) y sección de comentarios (b)



(a) (b)

Figura 24. Interfaces en Sprint 4: compartir publicación a aplicaciones externas (izquierda) y perfil de usuario (derecha)

Durante este sprint, se observó que el esfuerzo efectivo fue menor al estimado debido a PAWQ-13 como se muestra en la **Tabla *9***. Esta HU tuvo una implementación con menor esfuerzo gracias a la existencia de herramientas que facilitaron su ejecución, las cuales no se habían considerado al momento de estimar. Esto permitió generar un nuevo incremento funcional.

Tabla 9. Tabla comparativa de esfuerzo sprint 4

Historia de usuario	Esfuerzo estimado	Esfuerzo efectivo
PAWQ-14	55	55
PAWQ-13	55	35
PAWQ-30	21	21
PAWQ-128	5	5

2.5.5. Retrospectiva

En la sesión retrospectiva, se concluyó la efectividad de las pautas y estrategias de mejora tomadas en sprints anteriores y se decidió mantenerlas.

2.6 Sprint 5

2.6.1. Introducción

El objetivo del Sprint 5 es implementar la funcionalidad de autenticación del usuario y facilitar la comunicación con organizaciones con especialización en cuidado animal. Para asegurar un proceso de autenticación seguro, se decidió usar la tecnología JWT (JSON WEB TOKEN) que debe ser incluida en todas las peticiones para el manejo de sesiones y acceso a la API backend.

2.6.2. Planificación

En este sprint, se planeó integrar el sistema de autenticación y autorización usando JWT. Para lograrlo, se llevó a cabo una refactorización de las peticiones existentes para incluir el uso del token. Además, se ajustaron las interfaces para la obtención del token y se implementó un sistema de guardado para la gestión de estos en la memoria local. Las HU del sprint se muestran en la **Tabla *10***.

Tabla 10. Product Backlog Items sprint 5.

Identificador	Product backlog item	Estimación	Tareas
PAWQ-32	Como usuario registrado, quiero ingresar a la aplicación con mis credenciales para realizar publicaciones y gestionar adopciones de animales de compañía.	50	• Modificación de lógica de navegación y refactor para el manejo de JWT. • Crear una interfaz de “Inicio de Sesión” con un formulario de las credenciales del usuario.
PAWQ-31	Como usuario nuevo, quiero poder registrarme en la aplicación para crear una cuenta de usuario que me permita usar la aplicación.	45	• Crear interfaz “Registro” con un formulario que solicite datos del usuario.
PAWQ-33	Como usuario registrado, quiero poder modificar los datos de mi perfil para mantener mi información actualizada	35	• Botón actual “Editar Perfil” debe cambiarse de nombre porque solo edita campos de aptitud para esta tarea, se debe crear otra pantalla diferente y con otro botón distinto al anterior, y el de esta tarea si debe llamarse “Editar Perfil”.
PAWQ-16	Como promotor de la adopción responsable, quiero conocer sobre las plataformas de fundaciones y asociaciones de protección animal para estar informado.	89	• Prueba de concepto de APIs para acceder a redes sociales y web de organizaciones • Crear interfaz "Organizaciones" donde se liste diferentes plataformas de organizaciones importantes de Quito.

2.6.3. Implementación

Teniendo en cuenta la importancia de la seguridad, se desarrollaron funcionalidades clave derivadas del inicio de sesión y registro de usuario. Estas funcionalidades se abstrajeron y se reflejaron en los casos de uso presentados. Además, se creó una navegación dedicada para las interfaces de registro e inicio de sesión con el objetivo de mantener aislado el acceso a las funciones principales como se muestra en la **Figura *25***.

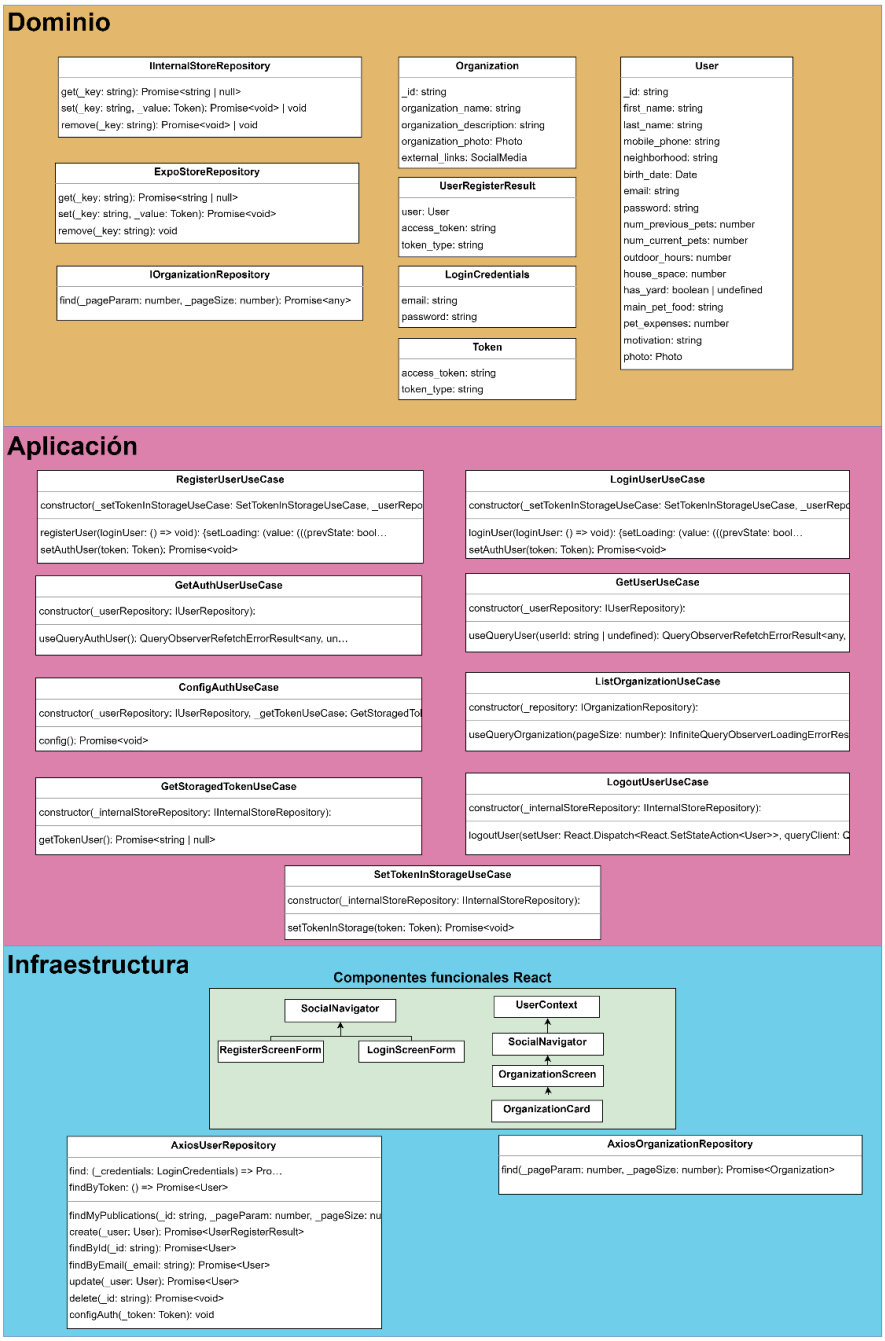


Figura 25. Arquitectura limpia para sprint 5

A continuación, se detalla el enfoque utilizado para cada PBI considerando las tareas relacionadas y los elementos mencionados en la **Figura *25***:

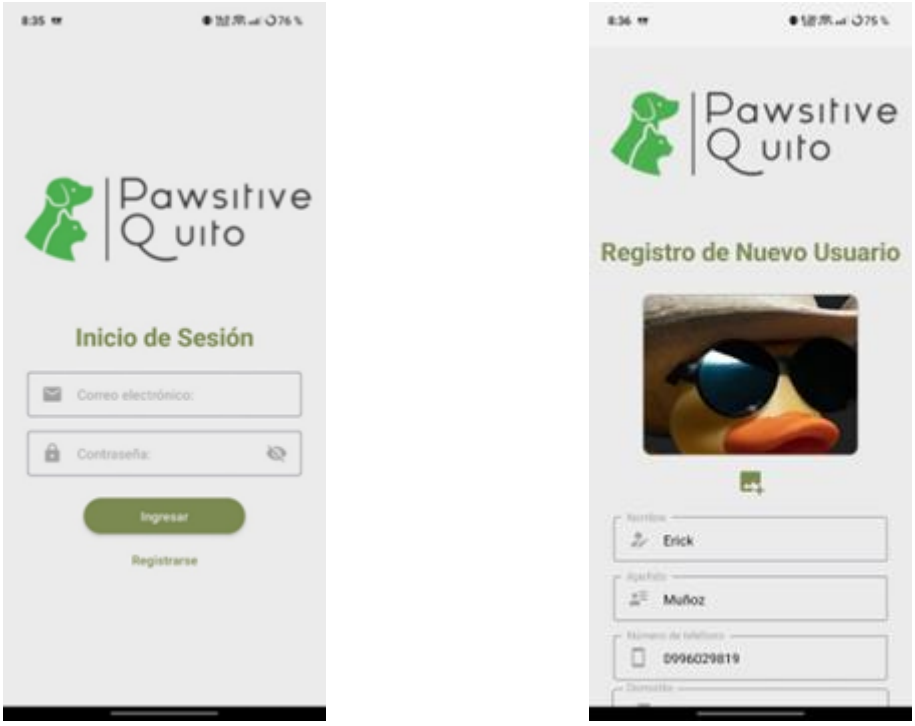
- PAWQ-32 Login: El proceso de inicio de sesión se desarrolló mediante la implementación de métodos faltantes en IUserRepository, incluyendo "find" para enviar datos de inicio de sesión (LoginCredentials) y obtener el token de autorización. Se crearon los casos de uso SetTokenInStorageUseCase y GetStoragedUseCase, utilizando IInternalRepository para definir cómo se guarda y

se obtiene el token en la memoria interna del dispositivo móvil. Los casos de uso LoginUseCase y LogoutUserUseCase se diseñaron para el inicio de sesión y cierre de sesión respectivamente, se utilizó los casos de uso mencionados junto con ConfigAuthUseCase para gestionar el token y configurar las peticiones, asegurando un inicio y cierre de sesión. Además, se incluyó una función para configurar el usuario que inicia sesión utilizando el hook useContext de React, permitiendo el acceso a esta información en distintos componentes. Para mantener la sesión activa, se extrajo el token de la memoria interna, lo que permitió realizar una petición para recuperar el usuario que inició sesión con anterioridad a través del caso de uso GetUserUseCase, el cual también usa el IUserRepository. Finalmente, los casos de uso mencionados se utilizan en los componentes LoginScreenForm y UserContext. También se introdujo ExpoStoreRepository para administrar el almacenamiento encriptado del token en la memoria interna del dispositivo móvil.

- PAWQ-31 Registro de Usuario: De la misma forma, en la interfaz de usuario RegisterScreenForm, se implementó el caso de uso RegisterUserUseCase. Este caso de uso emplea IUserRepository y otros casos de uso para garantizar el correcto manejo del token y las sesiones en el dispositivo móvil.
- PAWQ-33 Modificar datos de perfil: Para esta implementación, se finalizó la implementación de IUserRepository y se realizaron modificaciones en el componente ProfileScreen creado en sprint anteriores.
- PAWQ-16 Conocer plataformas de fundaciones/asociaciones de protección animal: En línea con la estructura de la arquitectura limpia, se estableció el componente OrganizationScreen. Este componente emplea del caso de uso ListOrganizationUseCase, que a su vez cuenta con un IOrganizationRepository, con su respectiva implementación en infraestructura. Esto se asemeja a la visualización de publicaciones definida en sprint anteriores, con la excepción de la creación de un nuevo componente llamado OrganizationCard. Este componente utiliza la información recibida para generar botones que permiten acceder a distintas redes sociales de cada asociación u organización de cuidado animal.

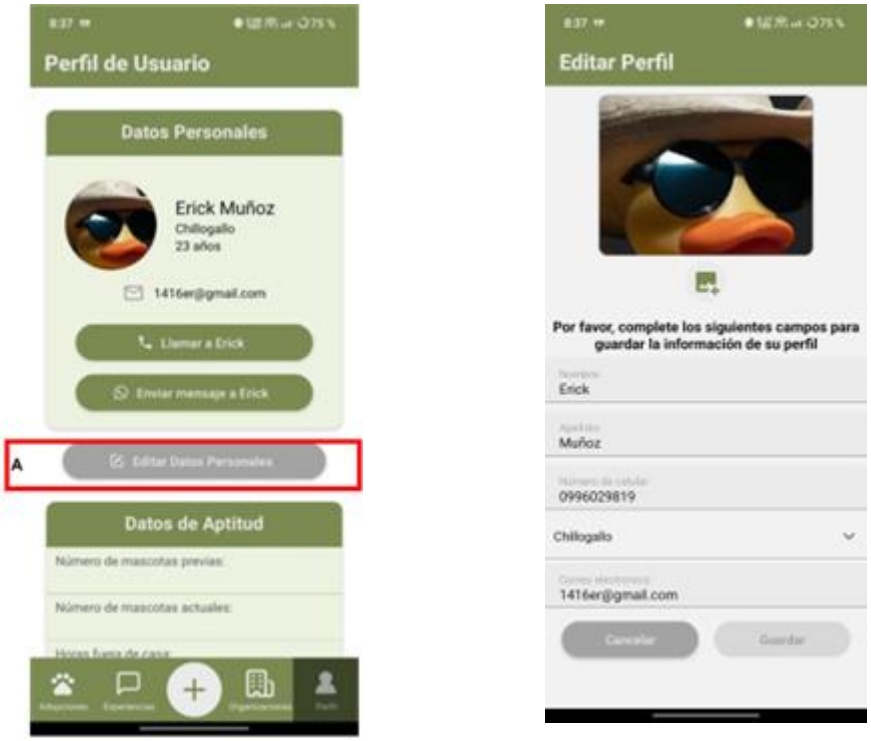
2.6.4. Revisión

En base a la implementación de la sección anterior se pueden notar los resultados en **Figura *26***, **Figura *27*** y **Figura *28***.



(a) (b)

Figura 26. Interfaces en Sprint 5: inicio de sesión (a) y registro de usuario (b)



(a) (b)

Figura 27. Interfaces en Sprint 5: botón para editar datos en A (a) y formulario de datos de usuario (b)

Figura 28. Interfaz en Sprint 5: organizaciones

Durante este sprint, las estimaciones de esfuerzo para los PBIs fueron precisas, como se muestra en la **Tabla *11*** . Esto nos permitió cumplir con los criterios del "Done", generando un incremento funcional en el proyecto.

Tabla 11. Tabla comparativa de esfuerzo sprint 5

Historia de usuario	Esfuerzo estimado	Esfuerzo efectivo
PAWQ-32	50	50
PAWQ-31	45	45
PAWQ-33	35	35
PAWQ-16	89	89

2.6.5. Retrospectiva

Manteniendo la técnica para identificar diferentes áreas de mejora, en la reunión de retrospectiva, se identificaron algunas áreas de mejora descritas a continuación:

1. Realizar una revisión de actividades relacionadas a los PBIs previo a la planificación del sprint (para evitar omitir las tareas de configuración y generación del archivo APK).

2. Realización de monitoreo en Sprint Backlog para evitar el descarte de PBIs.

2.7 Sprint 6

2.7.1. Introducción

En el Sprint 6 se ha trabajado en la generación del APK, la elaboración del manual de instalación y la realización de pruebas de usuario para evaluar la satisfacción de los usuarios finales. El objetivo es obtener una percepción general del usuario final sobre la aplicación.

2.7.2. Planificación

En la planificación del Sprint 6, se definieron PBIs clave (como se muestra en la **Tabla *12***) para la generación del APK utilizando Expo . Esto implicó configurar las variables de entorno y el formato de salida de la aplicación móvil. Además, se creó un manual de instalación para el archivo resultante. Para mantener consistencia con las pruebas de usuario, se desarrolló un protocolo que incluía la participación del usuario en la evaluación mediante el formulario SUS (System Usability Scale). Finalmente, se identificó un error en la sección de comentarios, y se abordó durante el presente sprint.

Tabla 12. Product Backlog Items sprint 6

Identificador	Product backlog item	Estimación	Tareas
PAWQ-143	Generación APK (Archivo de instalación para Android)	40	• Configuración de Expo para la generación del APK • Crear el manual de instalación para el APK • Generación del APK
PAWQ-145	Evaluación de la solución	89	• Creación del cuestionario SUS en Google forms. • Creación de protocolo de pruebas de usuario. • Realización de pruebas de usuario a 12 personas • Análisis de resultados obtenidos en las pruebas de usuario.
PAWQ-153	Error botón "Enviar" en comentarios	5	• Reconfiguración del botón enviar comentario.

2.7.3. Implementación

Considerando la importancia de la correcta configuración del entorno Expo para la construcción y generación del APK, se recolectó las credenciales necesarias para el acceso al API en la nube.

A continuación, se detalla el enfoque utilizado para cada PBI considerando las tareas relacionadas:

- PAWQ-143 Generación APK: Para la generación se realizó la actualización del SDK de Expo a la versión 49, de esta forma la configuración de las variables de entorno y de la generación de archivo de salida, se realizó en un archivo con formato JSON que especifica los datos mencionados, posteriormente se vinculó el entorno de Expo con una cuenta de desarrollador y se realizó la construcción (build) y generación del APK con las configuraciones previas.

- PAWQ-145 Evaluación de la solución: Se implementó un protocolo detallado en el **ANEXO *V***, el cual aborda tareas clave relacionadas con las funciones importantes de la aplicación. Este protocolo incluye la documentación de comentarios y la aplicación del cuestionario SUS. Se seleccionó un grupo de 12 usuarios finales, considerando los Perfil Persona previamente definidos, y se ejecutó el protocolo con cada participante. Posteriormente, se llevó a cabo un análisis de resultados para identificar patrones comunes en los comentarios de los usuarios. Utilizando el formato de cálculo establecido para el cuestionario SUS (descrito en la sección de resultados), se obtuvo un valor de 72 en la escala de SUS.

- PAWQ-153 Error “Botón Enviar” en comentarios: Ante una conexión fallida con el API, se observó que el botón de enviar comentario permanecía inactivo. Para abordar esto, fue necesario ajustar la lógica de la interfaz de comentarios y su vinculación con AddCommentUseCase. Cabe destacar que se preservó la estructura original durante este proceso.

2.7.4. Revisión

Para la revisión de este sprint se verificó la inclusión del manual de usuario, el APK la documentación asociada a las pruebas de usuario y cuestionario SUS, en el repositorio centralizado de Notion definido en el **ANEXO *VI***. Finalmente se tuvo que el esfuerzo estimado fue preciso como se muestra en la **Tabla *13***.

Tabla 13. Tabla comparativa de esfuerzo sprint 6

Historia de usuario	Esfuerzo estimado	Esfuerzo efectivo
PAWQ-143	40	40
PAWQ-145	89	89
PAWQ-153	5	5

2.7.5. Retrospectiva

En la retrospectiva, se resaltó la efectividad del monitoreo de los PBIs, destacando su impacto positivo en una sincronización más eficiente. No obstante, se identificaron áreas de mejora en relación con las pruebas de usabilidad. Se sugirió considerar la incorporación de grabaciones en lugar de documentación escrita para las pruebas realizadas, buscando así una mejora en la recopilación de datos. Asimismo, se propuso la consideración de grupos más numerosos para obtener una perspectiva más amplia y representativa.

3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

3.1 Resultados

Esta sección presenta los resultados derivados de las pruebas de usuario llevadas a cabo con la participación de 12 personas. Incluye tanto los comentarios recopilados durante estas evaluaciones como los resultados obtenidos del cuestionario SUS. El propósito principal es proporcionar una perspectiva del nivel de satisfacción experimentado por los usuarios finales con respecto al presente módulo.

Pruebas de usuario

Las pruebas de usuario se centraron en evaluar la satisfacción de los usuarios, es decir, en comprender cómo perciben el módulo al utilizarlo, para ello se les proporcionó el módulo y ciertos valores de entrada para la realización de tareas. Las tareas específicas que se llevaron a cabo para este propósito se muestran en la **Tabla *14*** y se detallan en el **ANEXO *V***.

Tabla 14. Tareas para pruebas de usuario

N. Tarea	Título
0	Inicio de Sesión
1	Realizar Publicación de adopción
2	Interactuar con Publicaciones de adopción

Mientras llevaban a cabo las tareas, los participantes proporcionaron comentarios a modo de retroalimentación. Estos comentarios fueron registrados en el repositorio de documentación en Notion definido en el **ANEXO *VI***. No obstante, se presenta un resumen de los comentarios organizados por áreas de mejora en la **Tabla 15**.

Tabla 15. Resumen de comentarios categorizados por áreas de mejora

Área de Mejora	**Recomendaciones**
Interfaz de Usuario y Navegación	• Reducir el tamaño de las barras superior e inferior. • Mejorar la disposición de filtros. • Añadir un buscador de parroquias en la sección correspondiente. • Clarificar la separación entre experiencias y adopciones.
Publicaciones y Comentarios	• Mostrar el número de comentarios en las publicaciones. • Indicar si la edad se refiere a meses o años en el registro. • Eliminar feedback innecesario en los comentarios.
Aspectos Visuales y Estéticos	• Cambiar colores para transmitir emociones más positivas.
Funcionalidades Específicas	• Evaluar el renombrar el nombre del botón “cortar” por "seleccionar" al seleccionar fotografías.
Recomendaciones/Funcionalidades Adicionales	• Permitir más de una foto por publicación. • Agregar opciones de reacción además de "like". • Implementar personalización visual, como modo oscuro. • Agregar sistema de calificación por confiabilidad de usuarios. • Permitir compartir publicaciones como enlaces para usuarios no registrados. • Incluir opción de donación para la aplicación. • Implementar opción de recortar imágenes en diferentes dimensiones.

Cabe destacar que, en este módulo, no se han implementado soluciones específicas para abordar los comentarios recopilados. Sin embargo, estos comentarios proporcionan una perspectiva de la percepción de satisfacción de los participantes.

Satisfacción de usuario

Como paso final de las pruebas realizadas, los participantes respondieron las 10 preguntas propuestas en el cuestionario SUS mediante una escala de Likert de 5 puntos, donde 1 es Totalmente en desacuerdo y 5 Totalmente de acuerdo . El resultado se muestra en **Figura 29.**

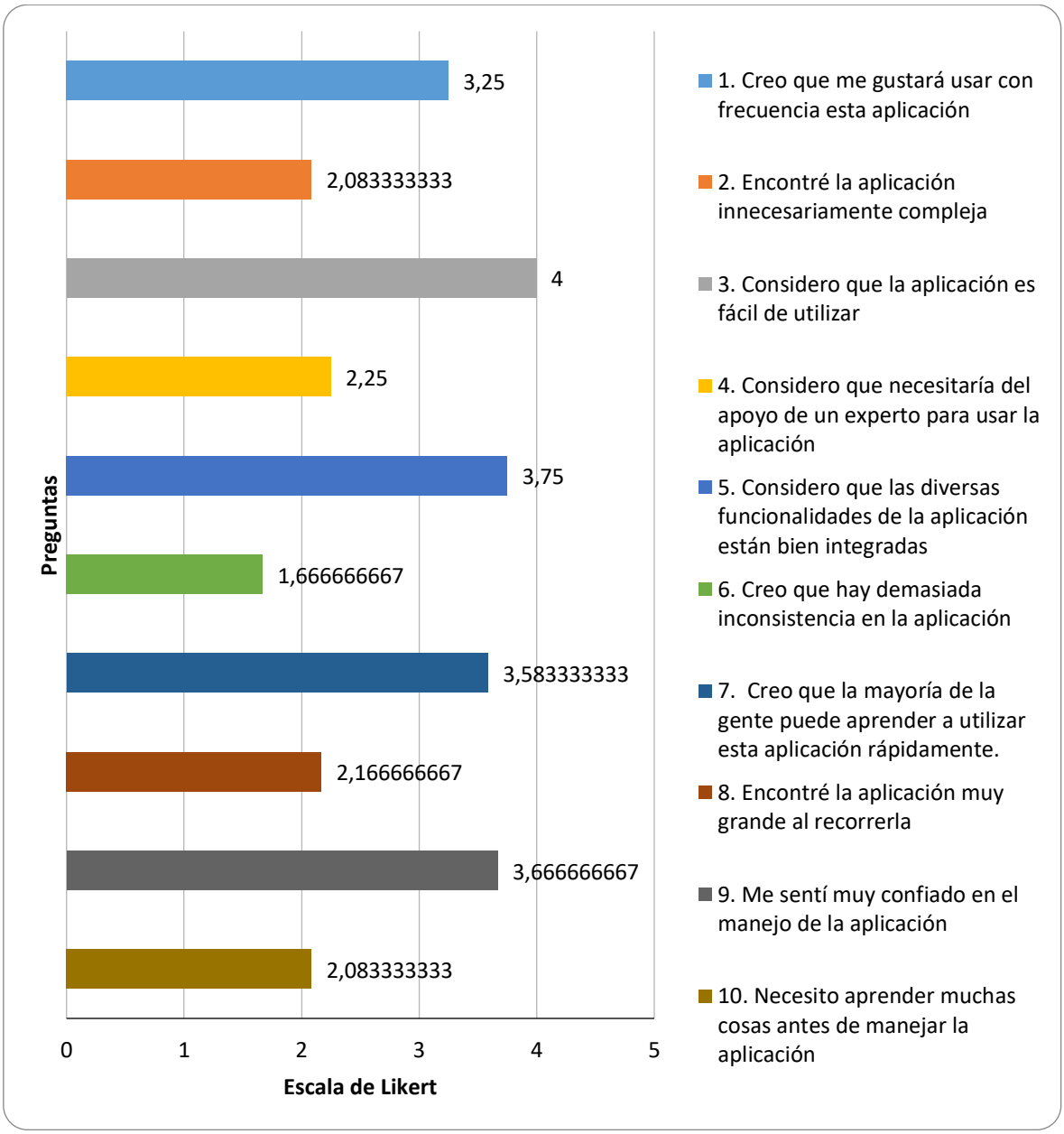


Figura 29. Puntaje promedio por pregunta - Cuestionario SUS

Se puede denotar una tendencia baja en las preguntas referentes a la complejidad e inconsistencia de la aplicación (preguntas 2,4,6,8,10) y una tendencia media-alta en las preguntas referentes a la confianza y facilidad de uso (preguntas 1,3,5,7,9).

Para obtener el resultado de la encuesta SUS, se aplica la siguiente estrategia definida en [24] para cada usuario:

I. Sea a el valor obtenido de las respuestas impares tal que:

$$a = \sum RespuestasImpares - 5$$

Ecuación 1. Cálculo de preguntas impares

II. Sea b el valor obtenido de las respuestas pares tal que:

$$b = 25 - \sum RespuestasPares$$

Ecuación 2. Cálculo de preguntas pares

III. Se R el valor resultante del SUS tal que:

$$R = (a + b) * 2.5$$

Ecuación 3. Cálculo del resultado SUS

De esta forma, se obtiene el puntaje SUS para cada participante, los valores obtenidos se muestran en **Tabla *16***.

Tabla 16. Puntaje SUS por participante

Participantes	**Puntaje SUS individual**
Sofia Soria	62,5
Steven Alvarado	57,5
Andrea Arevalo	82,5
Ramón Mendoza	65
Robinson Ortega	62,5
Santiago Suárez	57,5
Diana García	90
Daniel Encalada	90
Adrian Muñoz	75
Emily Ramos	47,5
Doménica Ramos	52,5
Jordy Manya	97,5

Finalmente, se realizó el promedio de los puntajes individuales para obtener un valor de 72, considerando la **Figura *30***, se puede mencionar que el presente módulo tiene una usabilidad aceptable.

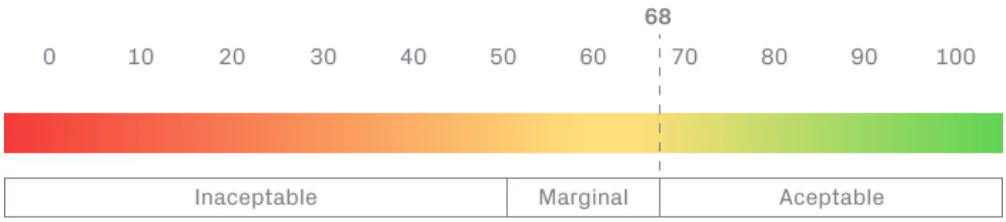


Figura 30. Representación de resultados SUS (tomada de [24])

3.2 Conclusiones

Considerando los objetivos específicos del presente módulo se muestran las conclusiones obtenidas a continuación:

Respecto al objetivo específico 1: La aplicación de técnicas del DCU condujo a una investigación de usuarios en el segmento de adopción a través de una encuesta y el análisis de perfiles utilizando el Perfil Persona. Con ello como base, se realizó el análisis de tareas mediante la matriz OTI que se tradujeron en tres épicas. Estos hallazgos se detallan en los primeros tres puntos en la sección 1 del SPRINT 0.

Respecto al objetivo específico 2: Se derivaron épicas del análisis de tareas, las cuales sirvieron de base para la creación de un prototipo inicial en la herramienta Figma. Este prototipo lo evaluaron un grupo de personas para obtener retroalimentación. Durante este proceso, se identificaron varios problemas de diseño, los cuales fueron documentados en el repositorio Notion definido en el **ANEXO *VI***. Estos problemas se utilizaron como insumo clave para el refinamiento del diseño en el desarrollo del SPRINT 1. El prototipo refinado se encuentra en el **ANEXO *VII***.

Respecto al objetivo específico 3: Considerando el prototipo refinado y las épicas definidas, se identificó que el resultado del módulo es impredecible, por lo cual, se optó por emplear el marco de trabajo SCRUM en el ciclo de vida del software del módulo. La implementación del módulo se llevó a cabo utilizando un enfoque de arquitectura limpia. Es relevante destacar que la elección de esta arquitectura fue crucial durante la aplicación de cambios y modificaciones en el módulo durante el desarrollo. Dichas modificaciones demostraron tener un impacto positivo, evidenciado por la obtención de un valor aceptable en el cuestionario SUS mencionado en la sección de satisfacción de usuarios en la sección Resultados, de este documento.

3.3 Recomendaciones

Esta sección proporciona las recomendaciones derivadas del desarrollo del módulo actual.

- Se recomienda realizar una planificación cuidadosa de las pruebas de usuario, asignando un número representativo de participantes y documentando de manera precisa los resultados obtenidos. Se sugiere incluir grabaciones de la ejecución de las pruebas con los usuarios, ya que esta práctica facilita la obtención de resultados más significativos y precisos, especialmente en la evaluación de la satisfacción de los usuarios finales.

- Es crucial señalar que, aunque la investigación de usuario puede identificar diversos requisitos para la solución del problema, se recomienda realizar un análisis de los recursos disponibles para el desarrollo. En base a este análisis, se sugiere seleccionar las funcionalidades que aporten valor al usuario final.

- Es crucial destacar que el refinamiento de prototipos antes de la implementación es una práctica esencial para optimizar la ejecución y el desarrollo del proyecto. Este enfoque permite identificar posibles mejoras y ajustes antes de comprometer recursos en la fase de implementación, contribuyendo así a un proceso más eficiente y exitoso.

- Se recomienda realizar un análisis exhaustivo para la selección de la arquitectura de software en proyectos frontend. Dado que estos proyectos están directamente expuestos al usuario y tienden a recibir un mayor número de solicitudes de cambios, es esencial optar por una arquitectura que garantice la escalabilidad y facilite la gestión eficiente de modificaciones. La elección de una arquitectura robusta y adaptable será fundamental para asegurar el éxito y la evolución positiva del proyecto.

- Es esencial tener en cuenta la definición de "terminado" (Done) al comenzar y concluir un sprint, especialmente en proyectos impredecibles. En entornos donde el software experimenta cambios y evoluciona constantemente, la definición de "terminado" no puede permanecer estática. Se recomienda adaptar y revisar periódicamente esta definición para asegurar que refleje de manera precisa los requisitos y estándares en evolución del proyecto.

4 REFERENCIAS BIBLIOGRÁFICAS

[1] I. Mantilla, “Más de 1 millón de perros y gatos deambulan por Quito”, *El Comercio*, ago. 2022.

[2] R. A. M. Viveros, “Percepción social del impacto de dos especies exóticas sobre fauna silvestre”, Universidad Veracruzana, 2020.

[3] Distrito Metropolitano de Quito, “ORDENANZA METROPOLITANA No. 019 - 2020”.

[4] ORGANIZACIÓN PANAMERICANA DE LA SALUD, *ZOONOSIS Y ENFERMEDADES*. Washington, DC, 2003.

[5] G. Coba, “Primicias - Más de un millón de perros y gatos habitan en las calles de Quito”.

[6] H. H. Herrera, “LAS REDES SOCIALES: UNA NUEVA HERRAMIENTA DE DIFUSIÓN”.

[7] Schwaber Ken y Sutherland Jeff, *La Guía Scrum*. 2020.

[8] ISO 9241, “Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts”.

[9] R. C. Martin, “Clean Architecture”.

[10] Codely, “Codely’s ESLint Hexagonal Architecture plugin”.

[11] C. Nance, *TypeScript Essentials*. 2014.

[12] B. Eisenman, “Learning React Native Building Mobile Applications with JavaScript”, 2016.

[13] Inc. Meta Platforms, “React Native - Cross Platform Implementation”, 2023.

[14] I. 650 Industries, “Expo - Tools and Services”.

[15] F. Flores, “Qué es Visual Studio Code y qué ventajas ofrece”, *OpenWebinars*, jul. 2022.

[16] B. Quiroga, “¿Qué es GIT y cómo funciona?”, EDTeam.

[17] G. D. Muriel y M. P. Enric, *Diseño centrado en el usuario*.

[18] Nacho Madrid, “Diseño Centrado en el Usuario: Pasado, presente y futuro”.

[19] Code Space, “Figma: qué es y cómo funciona”.

[20] Julia Martins, “Scrum: conceptos clave y cómo se aplica en la gestión de proyectos”, *Asana*, jun. 2023.

[21] K. Rubin, “Essential Scrum”.

[22] Cristhian Arias Venturo, “¿Cuándo aplicar Scrum? Una mirada al marco Cynefin”, *LinkedIn* , ene. 2017.

[23] Atlassian, “Bienvenido a Jira Software”.

[24] Cris Busquets, “Medir la usabilidad con el Sistema de Escalas de Usabilidad (SUS)”.

[25] Redacción KeepCoding, “Qué es SwiftUI”.

[26] IT Staff Augmentation RootStack, “React Native: Ventajas y desventajas de este framework”.

[27] Ionic Framework, “Introduction to Ionic”.

[28] Flutter y Google, “Flutter Docs”.

[29] Google for Developers, “Cómo comenzar a usar Android Jetpack”.

[30] IONOS, “Git vs. SVN: ¿cuál es el mejor sistema de control de versiones?”

[31] Mercurial SCM, “Work easier”.

[32] Microsoft, “Visual Studio Community”.

[33] Apache NetBeans, “Fits the Pieces Together”.

[34] Jet Brains, “Essential tools for software developers and teams”.

[35] Eclipse Foundation, “Eclipse Docs”.

[36] Microsoft, “Microsoft - Visual Studio Code”.

[37] Notion, “Notion -Wikis”.

[38] IONOS, “¿Qué es Jira? Todo lo que necesitas saber sobre el software de gestión de proyectos”.

5 ANEXOS

ANEXO I. Tabla comparativa de herramientas de desarrollo Frontend recopiladas de [25], [26], [27], [28], [29].

Característica	**React Native**	**Ionic**	**Flutter**	**SwiftUI**	**Android Jetpack**	**NativeScript**
Lenguajes de programación	JavaScript y Typescript	JavaScript	Dart	Swift	Java y Kotlin	JavaScript y TypeScript
Rendimiento nativo	Similar a aplicaciones nativas	Similar al de una aplicación web.	Similar a aplicaciones nativas.	Nativa IOS	Nativa Android	Similar a aplicaciones nativas
UI y experiencia de usuario	Componentes nativos.	Plantillas UI personalizables.	Widgets personalizados para crear UIs.	Sintaxis declarativa para crear UIs.	Bibliotecas para crear UIs.	Plantillas de diseño NativeScriptUI
Acceso a API y servicios nativos	Permite acceso a API nativas para los sistemas operativos Android e IOS.	El acceso a APIs y servicios nativos limitados.	Acceso a APIs y servicios nativos.	Acceso a APIs y servicios nativos. (solo Apple).	Acceso a APIs y servicios nativos. (solo Android).	Acceso a APIs y servicios nativos limitados.
Multiplataforma (iOS y Android)	Si	Si	Si	No	No	Si

Documentación y recursos de aprendizaje	Documentación oficial de React Native mantenida por Meta.	Documentación y comunidad que ofrecen recursos y soporte.	Documentación respaldada por Google.	Documentación respaldada por Apple.	Google proporciona documentación en Android Developers.	Documentación oficial de NativeScript.
Comunidad y soporte	Comunidad activa. (NPM).	Comunidad activa.	Comunidad activa.	Comunidad activa.	Comunidad activa.	Comunidad activa.
Herramientas y ecosistema	Integración con herramientas React y npm.	Plugins y librerías.	Plugins y librerías.	Xcode IDE de Apple.	Android Studio IDE de Google.	Soporte integrado para Angular, VueJS.

ANEXO II. Tabla comparativa de herramientas de Control de Versiones recopiladas de [16], [30], [31].

Característica	**Git**	**SVN**	**Mercurial**
Funcionalidades básicas	Creación y clonación de repositorios, creación y fusión de ramas, commits, etc.	Crear y clonar repositorios, crear y fusionar ramas, hacer commits, etc.	Creación y fusión de ramas, reversión de cambios
Arquitectura y modelo de datos	Arquitectura distribuida	Arquitectura centralizada	Arquitectura distribuida
Rendimiento	Alto rendimiento.	Baja velocidad en operaciones básicas.	Alto rendimiento.
Soporte para ramas y fusiones	Permite el trabajo en paralelo, ramas, fusiones automáticas, tags. etc.	Su modelo de datos lineal con gestión de ramas y fusiones compleja.	Capacidades avanzadas de ramificación y fusión.
Integración y compatibilidad	Compatible con GitHub, GitLab, BitBucket.	Compatible con GitHub, GitLab, BitBucket	Compatibilidad con GitHub y BitBucket. (usando extensiones).
Facilidad de uso y curva de aprendizaje	Curva de aprendizaje empinada.	Curva de aprendizaje media.	Curva de aprendizaje media.
Seguridad y control de acceso	Autenticación y autorización para el control de acceso a repositorios, encriptación de datos y firma digital de commits.	Autenticación y autorización para el control de acceso a repositorios, encriptación de datos y firma digital de commits.	Autenticación para el acceso a repositorios y encriptación de datos.
Comunidad y soporte	Gran comunidad de usuarios y desarrolladores.	Comunidad activa.	Comunidad activa que proporciona documentación y recursos de apoyo.

ANEXO III. Tabla comparativa de Editores/IDEs recopilados de [32], [33], [34], [35], [36]

Característica	VS Code	VS Community	Eclipse	WebStorm	NetBeans
Compatibilidad lenguajes de programación.	Alta	Alta	Media con plugins	Enfocado en desarrollo web.	Alta con plugins
Facilidad de uso y experiencia de usuario	Interfaz limpia, intuitiva, actualizaciones constantes.	Interfaz intuitiva con herramientas para desarrollo y depuración.	Interfaz personalizable, experiencia depende del lenguaje.	Interfaz intuitiva, familiar para usuarios JetBrains.	Interfaces sencillas e intuitivas.
Autocompletado y sugerencias de código	IntelliSense.	IntelliSense.	Predicción y sugerencias inteligentes.	Detecta contexto, predicción ágil.	Predicción y sugerencia según contexto.
Integración con herramientas externas/Soporte de complementos y extensiones	Mercado de extensiones.	Requiere experiencia para integración.	Diversas herramientas.	Fácil integración y SPACE Marketplace.	Diversas herramientas.
Rendimiento y consumo de recursos	Ligero consumo de recursos.	Alto consumo de recursos.	Medio consumo de recursos.	Mínimo consumo de recursos.	Mínimo consumo de recursos.

ANEXO IV. Tabla comparativa de herramientas de gestión de proyectos recopiladas de [37], [38].

Características	Jira	Trello	Azure DevOps	Notion
Seguimiento de tareas y gestión de proyectos	Amplias capacidades de seguimiento de tareas y gestión de proyectos.	Funciones básicas de seguimiento de tareas y gestión de proyectos.	Completo conjunto de herramientas para la gestión de proyectos.	Alta personalización y plantillas para la gestión de proyectos y tareas.
Tableros y vistas visuales	Tableros personalizables y vistas Kanban y Scrum.	Tableros Kanban visuales con enfoque en la simplicidad.	Tableros Kanban, tableros de sprint y vistas visuales personalizables.	Plantillas de Scrum, sprint y vistas visuales personalizables.
Colaboración y comunicación	Funcionalidades de colaboración y comunicación.	Comentarios y notificaciones básicas.	Funciones de colaboración y comunicación completas, incluyendo chat integrado.	Funciones de colaboración y comunicación nativas limitadas.

Integración con herramientas de versionamiento	Integración nativa con sistemas de control de versiones, como Git, bibucket, etc.	No tiene integración nativa.	Integración profunda con repositorios Git y otras herramientas de DevOps.	No tiene integración con herramientas de versionamiento nativa.
Gestión de dependencias	Posibilidad de gestionar dependencias y enlaces entre tareas.	No tiene una funcionalidad nativa para gestionar dependencias.	Soporte para gestionar dependencias y relaciones entre elementos de trabajo.	Soporte configurable y personalizable para gestionar dependencias y relaciones entre elementos.
Informes y métricas	Generación de informes y seguimiento de métricas.	Generación de informes y métricas.	Generación de informes y paneles de control personalizables.	No tiene control y generación de informes nativas.
Seguridad y control de acceso	Opciones de configuración de seguridad y control de acceso granulares.	Funcionalidad básica de seguridad y control de acceso.	Control de acceso y políticas de seguridad personalizables.	Control de acceso personalizable.
Comunidad y soporte	Gran comunidad y soporte de Atlassian.	Comunidad y soporte más limitados.	Amplia comunidad y soporte de Microsoft.	Amplia comunidad, Plantillas y recursos.
Enfoque de Proyectos	Equipos de desarrollo y gestión de proyectos complejos.	Ideal para equipos pequeños y proyectos sencillos.	Equipos que utilizan otras herramientas de Microsoft y Azure.	Ideal para la centralización de documentos.

ANEXO V. Protocolo para Pruebas de Usabilidad - PAWSITIVE

El protocolo de prueba de usabilidad tiene como objetivo evaluar la satisfacción del usuario al interactuar con la aplicación móvil PAWSITIVE, diseñada para promover el bienestar animal.

1. **Objetivo e Hipótesis:**
   1.1. **Objetivo:** Evaluar la satisfacción del usuario al interactuar con la aplicación móvil PAWSITIVE, diseñada para promover el bienestar animal.
   1.2. **Hipótesis:**
      1.2.1. Ho: La satisfacción percibida por el usuario al interactuar con la aplicación PAWSITIVE es mayor o igual a 68 puntos de la escala SUS.
2. **Variable Dependiente y Métrica:**
   2.1. Satisfacción Global (SUS):
      1. Métrica: Puntuación en el Cuestionario de Usabilidad del Sistema (SUS) consistente de 10 preguntas.
2. **Participantes:**

Se reclutarán 12 personas considerando los 3 perfiles Persona definidos en el DCU.

3. **Logística:**
   - Coordinar el lugar para la ejecución de la prueba con los participantes.

- Proveer un dispositivo móvil Android con acceso a la aplicación PAWSITIVE.
- Ejecutar la prueba en modo presencial, en un entorno controlado.

4. Materiales Necesarios:

- Dispositivos móviles con sistema operativo Android y con la aplicación PAWSITIVE instalada.
- Área designada para las pruebas.
- Conexión a internet
- Instrucciones detalladas para cada tarea.

5. Tareas:

Para evaluar la hipótesis Ho, se plantea un escenario de 4 tareas:

Escenario: “Eres una persona que está interesada en dar en adopción un animal de compañía y aprender sobre los cuidados y responsabilidad que conlleva tener uno. Utilizando la aplicación PAWSATIVE realiza las siguientes tareas (el encargado de la prueba no podrá guiar al usuario, solo brindar indicaciones generales).

Tarea 0: Inicio de Sesión

- Ingresar a la pantalla de inicio de sesión
- Ingresar las credenciales provistas por el encargado de la prueba.

Tarea 1: Ejecución del Juego:

- Ingresar a la aplicación.
- Navegar al módulo de juegos.
- Seleccionar el juego “Leyes y Sanciones” hasta terminar la partida (ganar o perder).

Tarea 2: Realizar Publicación:

- Ingresar a la aplicación.
- Acceder al formulario para agregar una publicación de adopción.
- Llenar la información con los siguientes datos:
  - Añadir la fotografía de un animal de compañía desde la galería
  - **Especie:** Perro
  - **Sexo:** Macho
  - **Raza:** Mestizo
  - **Edad:** 11 meses
  - **Tamaño del animal:** Mediano
  - **Sector:** San Bartolo
  - **Datos de esterilización y vacunación:** No esta esterilizado y Sí tiene carnet de vacunación
  - **Descripción: “**perro llamado Firulais, es bonito y amistoso busca hogar”
- Publicar y visualizar la publicación aplicando los siguientes filtros:
  - Perro
  - San Bartolo
  - La fecha del día actual.

Tarea 3: Interactuar con Publicaciones:

- Ingresar a la aplicación.
- Visualizar la publicación más reciente.
- Interactuar con la publicación (dar "like", comentar, compartir).

- Guardar y visualizar la publicación en favoritos.

6. **Procedimiento:**
   1. Introducción
      1.1. Recibir al participante e indicar el objetivo de la prueba
      1.2. Brindar al usuario contexto sobre la aplicación PAWSITIVE
      1.3. Solicitar al participante consentimiento para usar los datos de la prueba para fines académicos.
      1.4. Proporcionar al participante el documento escrito de las tareas y las instrucciones detalladas para realizar cada tarea.
      1.5. Solicitar al participante explorar la interfaz para familiarizarse.
      1.6. Solicitar a los participantes comunicar sus pensamientos en voz alta (think aloud) mientras completan las tareas.
   2. Iniciar la prueba (ejecución de tareas).
   3. Solicitar al participante comentarios generales sobre la aplicación.
   4. Solicitar al participante completar el Cuestionario SUS.
   5. Agradecer la participación en la prueba.

ANEXO VI. Enlace a la documentación centralizada - Notion

https://forested-rhubarb-1ab.notion.site/Documentaci-n-TIC-2023-GMC-bb5af20f6b2d499e9c19ae16fa791f48

ANEXO VII. Enlace al prototipo Figma refinado

https://www.figma.com/file/KRDTRRu5dSqqqWy7EXUPUh/MockUps---Pawsitive-Quito?type=design&node-id=0%3A1&mode=design&t=nez1P4yuuppfKJgx-1

ANEXO VIII. Cuestionario SUS: PAWSITIVE-QUITO

Mediante el método SUS (Sistema de Escalas de Usabilidad, System Usability Scale) el presente cuestionario busca analizar la experiencia de usuario de los participantes de las pruebas de usuario de la aplicación móvil "PAWSITIVE-QUITO". Las respuestas corresponden a la escala de Likert con las siguientes opciones:

1 (Totalmente en desacuerdo)

2 (En desacuerdo)

3 (Ni de acuerdo ni en desacuerdo)

4 (De acuerdo)

5 (Totalmente de acuerdo)

Indicaciones

A continuación, se presentará una serie de enunciados con los que se debe indicar el grado de acuerdo o desacuerdo que se tiene con cada enunciado.

Ingrese su nombre _____

Ingrese su apellido _____

1. Creo que me gustará usar con frecuencia esta aplicación

2. Encontré la aplicación innecesariamente compleja

3. Considero que la aplicación es fácil de utilizar

4. Considero que necesitaría del apoyo de un experto para usar la aplicación

5. Considero que las diversas funcionalidades de la aplicación están bien integradas

6. Creo que hay demasiada inconsistencia en la aplicación

7. Creo que la mayoría de la gente puede aprender a utilizar esta aplicación rápidamente.

8. Encontré la aplicación muy grande al recorrerla

9. Me sentí muy confiado en el manejo de la aplicación

10. Necesito aprender muchas cosas antes de manejar la aplicación

ANEXO IX. Encuesta sobre adopción, bienestar animal y tenencia responsable en el DMQ.

Esta encuesta está realizada por estudiantes de la Escuela Politécnica Nacional como parte de un proyecto de Trabajo de Integración Curricular y tiene como objetivo recolectar información sobre la percepción y conocimientos que tienen las personas residentes en el Distrito Metropolitano de Quito con respecto a:

- La adopción de perros y gatos
- El bienestar animal y la tenencia responsable

Le solicitamos de favor leer detenidamente las preguntas y contestar todas las preguntas con honestidad (no se solicitan datos personales). Recuerde que tanto los datos, como resultados de la encuesta, serán usados con fines académicos.

Datos demográficos

En esta sección se solicitan datos demográficos para poder identificar segmentos de la población que responda la encuesta, además de conocer ciertos aspectos de su estilo de vida actual para enlazarlos con las preguntas de secciones diferentes.

1. Selecciona su rango de edad
    - Menos de 18 años
    - Entre 18 y 25 años
    - Entre 26 y 35 años
    - Más de 35 años
2. Seleccione su sexo
    - Femenino
    - Masculino
3. ¿En qué sector de Quito reside actualmente?
    - Quito Sur
    - Quito Centro
    - Quito Norte
    - Valles (Cumbayá, Los Chillos)
4. ¿Con quién vive actualmente?
    - Solo/a
    - Con familiares
    - Con amigos o compañeros de piso
    - Con pareja
5. ¿Cuál es tu ocupación principal actual?
    - Estudiante
    - Empleado dependiente
    - Empleado autónomo
    - No trabajo ni estudio

- Jubilado

Experiencia en Adopción

En esta sección se busca conocer qué experiencia ha tenido con respecto al tema de la adopción de perros y gatos. También se busca establecer su interés sobre una aplicación que permita apoyar el proceso de adopción.

6. ¿Cuántas veces has adoptado un perro o un gato en los últimos años?
    - Una vez
    - Varias veces
7. ¿Cuál fue tu principal motivo para adoptar?
    - Amor hacia los animales.
    - Brindar hogar a un animal necesitado.
    - Compañía y afecto de una mascota.
    - Inspiración por experiencias de Familiares o Amigos.
    - Ninguna de las anteriores
8. ¿Cuál es el principal método de adopción que conoce?
    - De forma particular (mediante redes sociales, eventos, familiares, amigos)
    - Mediante un refugio de animales / asociaciones de rescate
    - Tienda de animales
9. ¿Cuál fue tu experiencia con el proceso de adopción?
    - Muy satisfactoria
    - Satisfactoria
    - Neutral
    - Poco satisfactoria
    - Nada satisfactoria
10. ¿Cuál es la principal característica que usted considera al momento de adoptar un perro o un gato?

- Apariencia física (Tamaño, Raza, Sexo)
- Edad
- Comportamiento
- Historial médico (enfermedades, vacunas, desparasitación)
- Si está esterilizado o no
- Otro

11. ¿Cuáles de los siguientes aspectos considera al momento de realizar una adopción? (Puede seleccionar más de uno)
    - Dinero
    - Tiempo
    - Espacio
12. ¿Conoce alguna aplicación relacionada con la adopción de animales?
    - Si
    - No
13. ¿Estaría interesado/a en aplicación relacionada con la adopción de animales?
    - Si
    - No

Uso de redes sociales

En esta sección se explora el nivel de uso de las redes sociales, por parte de las personas, en relación con la adopción de perros o gatos. Estas preguntas tienen el objetivo de obtener información sobre el papel de las redes sociales en la búsqueda de hogares para perros y gatos y la difusión de información relacionada con la adopción y el cuidado de animales.

14. ¿Con qué frecuencia utiliza redes sociales?
    - Varias veces al día.
    - Al menos una vez al día.
    - Al menos una vez a la semana.

- Ocasionalmente, solo para verificar noticias o actualizaciones.
- No utilizo redes sociales.

15. Al dar en adopción un perro o un gato ¿Cómo le encontraría un hogar?
    - Publicaciones en redes sociales
    - Contacto directo con familiares o amigos
    - Eventos / Programas de adopción
    - Otro
16. ¿Ha compartido o compartiría experiencias, consejos e ideas sobre temas de adopción y cuidado de los animales con otras personas?
    - Si
    - No

Tenencia responsable

En esta sección se tiene como objetivo recolectar información sobre el conocimiento, familiaridad y percepción de las personas sobre el bienestar animal, las responsabilidades y sanciones asociadas a la tenencia responsable.

17. ¿Qué aspectos considera importantes para el cuidado de un perro o gato? (Puede seleccionar más de uno)
    - Información detallada sobre su cuidado y necesidades.
    - Programas de capacitación en comportamiento y entrenamiento de animales.
    - Servicios veterinarios.
    - Comunidades de apoyo relacionadas al cuidado de perros y gatos.
18. ¿Conoce las "5 libertades del bienestar animal"?
    - Si
    - No
19. ¿Tiene conocimiento de las leyes/sanciones relacionadas al bienestar animal que aplican en el país?
    - Si

- No

20. ¿Cuáles consideras que son las responsabilidades que debe cumplir una persona que cuida de un animal?

    - Educarlo y ejercitarlo, Protegerlo del sufrimiento y miedo, Responder por los daños y perjuicios que pueda causar
    - Amar al animal, Esterilizarlo, Acompañarlo en todo momento, No maltratarlo, Registrarlo oficialmente

ANEXO X. Manual de instalación de PAWSITIVE-QUITO

Pasos para instalar PAWSITIVE-QUITO en Android:

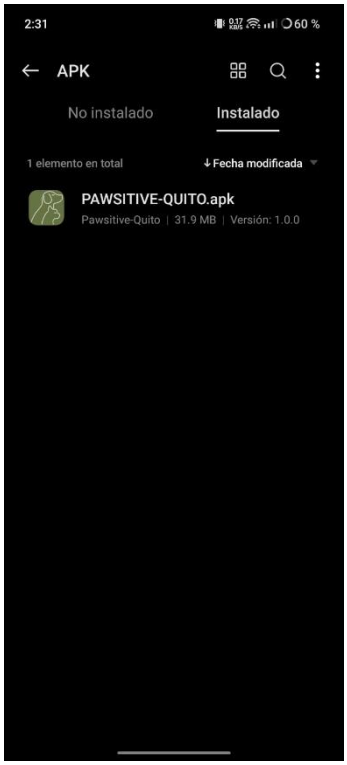
NOTA: Los pasos a seguir pueden tener variaciones según la versión de Android o la marca y modelo del dispositivo. El siguiente ejemplo se presenta en un dispositivo móvil OnePlus 10T con Android 13 (Oxygen OS), aunque se sugiere utilizar Android 12 o superior.

1. **Descarga el archivo APK:**

Encuentra el archivo APK que deseas instalar. Puedes descargarlo desde el navegador en tu dispositivo o transferirlo desde tu computador a tu teléfono a través de USB o mediante servicios en la nube. Desde el siguiente enlace PAWSITIVE-QUITO.apk

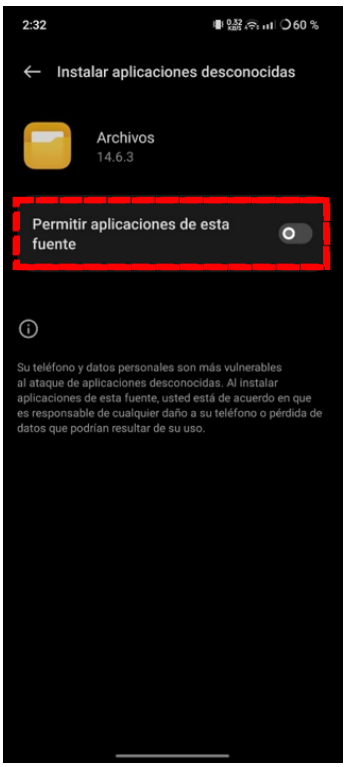
2. **Ubica el archivo APK:**

Abre la aplicación "Archivos" o el administrador de archivos en tu dispositivo Android. Navega hasta la ubicación donde se encuentra el archivo APK descargado o transferido y da clic sobre él.



3. **Habilita la instalación:**

Si es la primera vez que se instala una APK, es posible que se pida permiso para instalarla desde esta fuente. Confirma la instalación de la aplicación dando clic en **configuración** y sigue las instrucciones en pantalla como se muestra a continuación **(se debe otorgar el permiso en el recuadro rojo).**

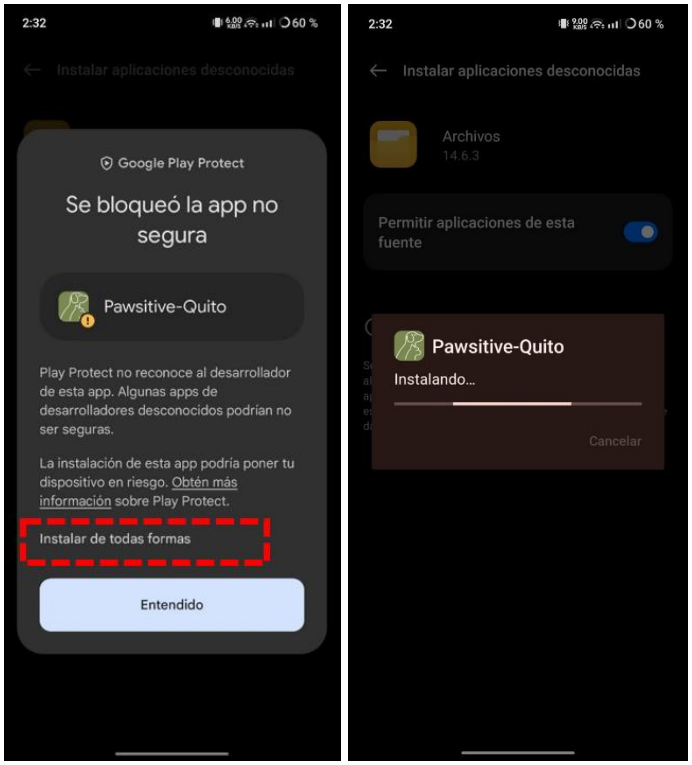


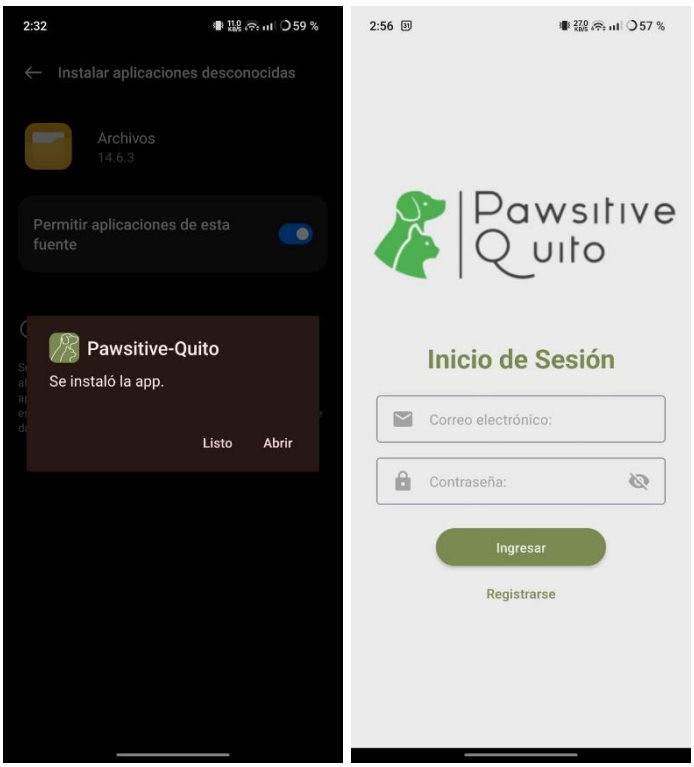
4. **Instalación del APK:**

Se abrirá una ventana para la instalación de forma automática, sino ocurre, da clic sobre el archivo APK nuevamente. Selecciona **"Instalar"** para comenzar con el proceso de instalación.

5. **Permiso de instalación:**

Es posible que se bloque la instalación, para ello es necesario seleccionar la opción de **“Instalar de todas formas”**. Como se muestra a continuación en el recuadro rojo:



6. **Finalización de la instalación:**

Una vez completada la instalación, se mostrará un mensaje que indica **"Se instaló la app"** o similar. Puedes optar por abrir la aplicación inmediatamente o regresar a la pantalla de inicio. Finalmente tendrás la aplicación instalada y podrás abrirla por primera vez.